Lesson 6: Regression Shrinkage Methods

Code

RIDGE REGRESSION THE LASSO

Overview

Prediction:

- Linear regression: $E(Y_j|X) = X\beta$;
- Or for a more general regression function: $E(Y_j|X) = f(X)$;
- In a prediction context, there is less concern about the values of the components of the right-hand side, rather interest is on the total contribution.

Variable Selection:

- The driving force behind variable selection:
  - The desire for a parsimonious regression model (one that is simpler and easier to interpret);
  - The need for greater accuracy in prediction.
- The notion of what makes a variable "important" is still not well understood, but one interpretation (Breiman, 2001) is that a variable is important if dropping it seriously affects prediction accuracy.
- Selecting variables in regression models is a complicated problem, and there are many conflicting views on which type of variable selection procedure is best, e.g. LRT, *F*-test, AIC, and BIC.

There are two main types of stepwise procedures in regression:

- Backward elimination: eliminate the least important variable from the selected ones.
- Forward selection: add the most important variable from the remaining ones.
- A hybrid version that incorporates ideas from both main types: alternates backward and forward steps, and stops when all variables have either been retained for inclusion or removed.

Criticisms of Stepwise Methods:

- There is no guarantee that the subsets obtained from stepwise procedures will contain the same variables or even be the "best" subset.
- When there are more variables than observations ($p > n$), backward elimination is typically not a feasible procedure.
- The maximum or minimum of a set of correlated *F* statistics is not itself an *F* statistic.

- It produces a single answer (a very specific subset) to the variable selection problem, although several different subsets may be equally good for regression purposes.
- The computing is easy by the use of R function

`step()` or `regsubsets()`. However, to specify a practically good answer, you must know the practical context in which your inference will be used.

Scott Zeger on 'how to pick the wrong model': Turn your scientific problem over to a computer that, knowing nothing about your science or your question, is very good at optimizing AIC, BIC, …

Objectives

Upon successful completion of this lesson, you should be able to:

- Introducing biased regression methods to reduce variance.
- Implementation of Ridge and Lasso regression.

6.1: Ridge Regression

Motivation: too many predictors

- It is not unusual to see the number of input variables greatly exceed the number of observations, e.g. microarray data analysis, environmental pollution studies.
- With many predictors, fitting the full model without penalization will result in large prediction intervals, and LS regression estimator may not uniquely exist.

Motivation: ill-conditioned X

- Because the LS estimates depend upon $(X'X)^{-1}$, we would have problems in computing β_{LS} if $X'X$ were singular or nearly singular.
- In those cases, small changes to the elements of X lead to large changes in $(X'X)^{-1}$.
- The least square estimator β_{LS} may provide a good fit to the training data, but it will not fit sufficiently well to the test data.

Ridge Regression:

One way out of this situation is to abandon the requirement of an unbiased estimator.

We assume only that X's and Y have been centered so that we have no need for a constant term in the regression:

- X is an *n* by *p* matrix with centered columns,

- Y is a centered n-vector.

Hoerl and Kennard (1970) proposed that potential instability in the LS estimator

$$\hat{\beta} = (X'X)^{-1}X'Y,$$

could be improved by adding a small constant value λ to the diagonal entries of the matrix $X'X$ before taking its inverse.

The result is the ridge regression estimator

$$\hat{\beta}_{ridge} = (X'X + \lambda I_p)^{-1}X'Y$$

Ridge regression places a particular form of constraint on the parameters (β's): $\hat{\beta}_{ridge}$ is chosen to minimize the penalized sum of squares:

$$\sum_{i=1}^{n}(y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$

which is equivalent to minimization of $\sum_{i=1}^{n}(y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2$ subject to, for some $c > 0$, $\sum_{j=1}^{p}\beta_j^2 < c$, i.e. constraining the sum of the squared coefficients.

Therefore, ridge regression puts further constraints on the parameters, β_j's, in the linear model. In this case, what we are doing is that instead of just minimizing the residual sum of squares we also have a penalty term on the β's. This penalty term is λ *(a pre-chosen constant)* times the squared norm of the β vector. This means that if the β_j's take on large values, the optimization function is penalized. We would prefer to take smaller β_j's, or β_j's that are close to zero to drive the penalty term small.

Geometric Interpretation of Ridge Regression:

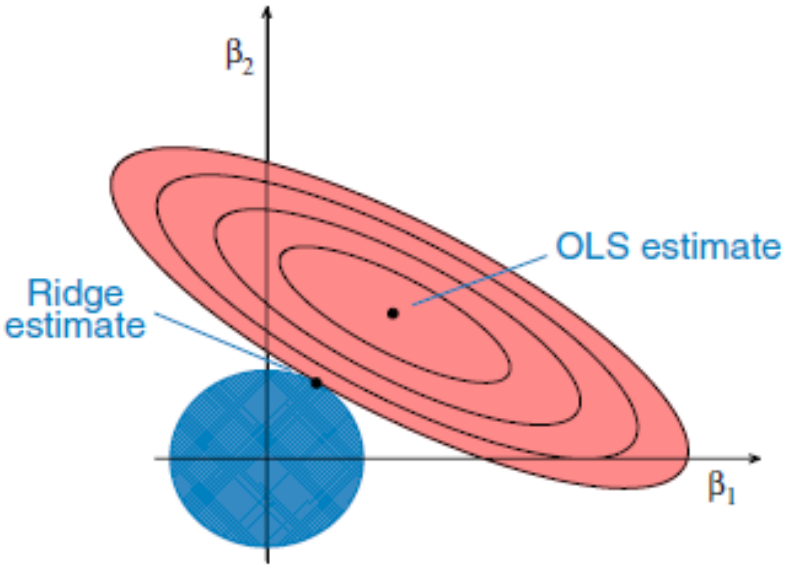


Fig 6.1

The ellipses correspond to the contours of the residual sum of squares (RSS): the inner ellipse has smaller RSS, and RSS is minimized at ordinal least square (OLS) estimates.

For $p = 2$, the constraint in ridge regression corresponds to a circle, $\sum_{j=1}^{p}\beta_j^2 < c$.

We are trying to minimize the ellipse size and circle simultaneously in the ridge regression. The ridge estimate is given by the point at which the ellipse and the circle touch.

There is a trade-off between the penalty term and RSS. Maybe a large β would give you a better residual sum of squares but then it will push the penalty term higher. This is why you might actually prefer smaller β's with a worse residual sum of squares. From an optimization perspective, the penalty term is equivalent to a constraint on the β's. The function is still the residual sum of squares but now you constrain the norm of the β_{j*}'s to be smaller than some constant c*. There is a correspondence between* λ and c*. The larger the λ is, the more you prefer the β_j's close to zero. In the extreme case when $\lambda = 0$, then you would simply be doing a normal linear regression. And the other extreme as λ approaches infinity, you set all the β's to zero.

Properties of Ridge Estimator:

- $\hat{\beta}_{ls}$ is an unbiased estimator of β; $\hat{\beta}_{ridge}$ is a biased estimator of β.

For orthogonal covariates, $X'X = nI_p$, $\hat{\beta}_{ridge} = \frac{n}{n+\lambda}\hat{\beta}_{ls}$. Hence, in this case, the ridge estimator always produces shrinkage towards 0. λ controls the amount of shrinkage.

An important concept in shrinkage is the "effective'' degrees of freedom associated with a set of parameters. In a ridge regression setting:

1. If we choose $\lambda = 0$, we have p parameters (since there is no penalization).
2. If λ is large, the parameters are heavily constrained and the degrees of freedom will effectively be lower, tending to 0 as $\lambda \to \infty$.

The effective degrees of freedom associated with $\beta_1, \beta_2, \ldots, \beta_p$ is defined as

$$df(\lambda) = tr(X(X'X + \lambda I_p)^{-1}X') = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda},$$

where d_j are the singular values of X. Notice that $\lambda = 0$, which corresponds to no shrinkage, gives $df(\lambda) = p$ (as long as $X'X$ is non-singular), as we would expect.

There is a 1:1 mapping between λ and the degrees of freedom, so in practice one may simply pick the effective degrees of freedom that one would like associated with the fit, and solve for λ.

- As an alternative to a user-chosen λ, cross-validation is often used in choosing λ: we select λ that yields the smallest cross-validation prediction error.
- The intercept β_0 has been left out of the penalty term because Y has been centered. Penalization of the intercept would make the procedure depend on the origin chosen for Y.
- Since the ridge estimator is linear, it is straightforward to calculate the variance-covariance matrix $var(\hat{\beta}_{ridge}) = \sigma^2(X'X + \lambda I_p)^{-1}X'X(X'X + \lambda I_p)^{-1}$.

A Bayesian Formulation

Consider the linear regression model with normal errors:

$$Y_i = \sum_{j=1}^{p} X_{ij}\beta_j + \epsilon_i$$

ϵ_i is i.i.d. normal errors with mean 0 and known variance σ^2.

Since λ is applied to the squared norm of the β vector, people often standardize all of the covariates to make them have a similar scale. Assume β_j has the prior distribution $\beta * j* \sim iidN(0, \sigma^2/\lambda)$. A large value of λ corresponds to a prior that is more tightly concentrated around zero and hence leads to greater shrinkage towards zero.

The posterior is $\beta|Y \sim N(\hat{\beta}, \sigma^2(X'X + \lambda I_p)^{-1}X'X(X'X + \lambda I_p)^{-1})$, where $\hat{\beta} = \hat{\beta}_{ridge} = (X'X + \lambda I_p)^{-1}X'Y$, confirming that the posterior mean (and mode) of the Bayesian linear model corresponds to the ridge regression estimator.

Whereas the least squares solutions $\hat{\beta}_{ls} = (X'X)^{-1}X'Y$ are unbiased if model is correctly specified, ridge solutions are biased, $E(\hat{\beta}_{ridge}) \neq \beta$. However, at the cost of bias, ridge regression reduces the variance, and thus might reduce the mean squared error (MSE).

$$MSE = Bias^2 + Variance$$

More Geometric Interpretations (optional)

$$\begin{aligned}\hat{y} &= \mathbf{X}\hat{\beta}^{ridge} \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \\ &= \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^T\mathbf{y} \\ &= \sum_{j=1}^{p}\mathbf{u}_j\frac{d_j^2}{d_j^2 + \lambda}\mathbf{u}_j^T\mathbf{y}\end{aligned}$$

where $\mathbf{u}_j$ are the normalized principal components of **X**.

$$\hat{\beta}_j^{ridge} = \frac{d_j^2}{d_j^2 + \lambda}\mathbf{u}_j^T\mathbf{y}$$

$$Var(\hat{\beta}_j) = \frac{\sigma^2}{d_j^2}$$

where σ^2 is the variance of the error term ϵ in the linear model.

- Inputs are centered first;
- Consider the fitted response
- Ridge regression shrinks the coordinates with respect to the orthonormal basis formed by the principal components.
- Coordinates with respect to principal components with smaller variance are shrunk more.
- Instead of using $X = (X_1, X_2, \ldots, X_p)$ as predicting variables, use the new input matrix $\tilde{X}$ = **UD**
- Then for the new inputs:

- The shrinkage factor given by ridge regression is:

$$\frac{d_j^2}{d_j^2 + \lambda}$$

We saw this in the previous formula. The larger λ is, the more the projection is shrunk in the direction of u_j. Coordinates with respect to the principal components with a smaller variance are shrunk more.

Let's take a look at this geometrically.

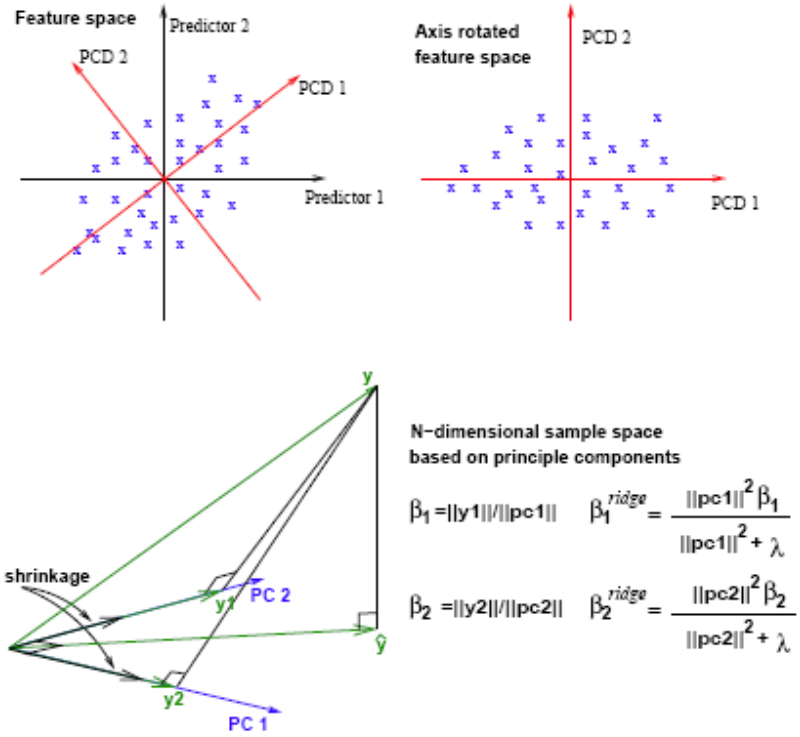


Figure 2: The Geometric interpretation of principal components and shrinkage by ridge regression.

Fig 6.2

View the Video Explanation



Shrinkage

This interpretation will become convenient when we compare it to principal components regression where instead of doing shrinkage, we either shrink the direction closer to zero or we don't shrink at all. We will see this in the Dimension Reduction Methods lesson.

6.2: Compare Squared Loss for Ridge Regression

Let's compare squared loss with and without ridge regression, i.e., $E(\beta_j - \hat{\beta}_j)^2$.

Without shrinkage we know that $\hat{\beta}_j$ has a mean of β_j since $\hat{\beta}_j$ is actually an unbiased estimator. Therefore, the expectation of the squared difference between β_j and $\hat{\beta}_j$ is simply the variance of $\hat{\beta}_j$, which is given by σ^2/d_j^2 .

But, if we use shrinkage the estimates are no longer unbiased. Therefore, this expectation becomes the Bias^2 + Variance of the ridge regression coefficient:

$$\left(\beta_j - \beta_j \cdot \frac{d_j^2}{d_j^2+\lambda}\right)^2 + \frac{\sigma^2}{d_j^2} \cdot \left(\frac{d_j^2}{d_j^2+\lambda}\right)^2 = \frac{\sigma^2}{d_j^2} \cdot \frac{d_j^2\left(d_j^2 + \lambda^2\frac{\beta_j^2}{\sigma^2}\right)}{(d_j^2+\lambda)^2}$$

You can see that after all of this calculation the ratio of expected squared loss between linear regression and ridge regression is given by this factor:

$$\frac{d_j^2\left(d_j^2 + \lambda^2\frac{\beta_j^2}{\sigma^2}\right)}{(d_j^2+\lambda)^2}$$

If this factor is more than one, this means that ridge regression gives, on average, more squared loss as compared to linear regression. In other words, if this factor is greater than one then ridge regression is not doing a good job.

This factor depends on a lot of things. It depends on λ, d_j , and the ratio β_j^2/σ^2.

Here is a plot where λ is fixed at 1. The ratio between β^2 and σ^2 is set on the specific values, 0.5, 1.0, 2.0 and 4.0. Then we plotted the squared loss ratio on the *y*-axis and the *x*-axis is d_j^2 .

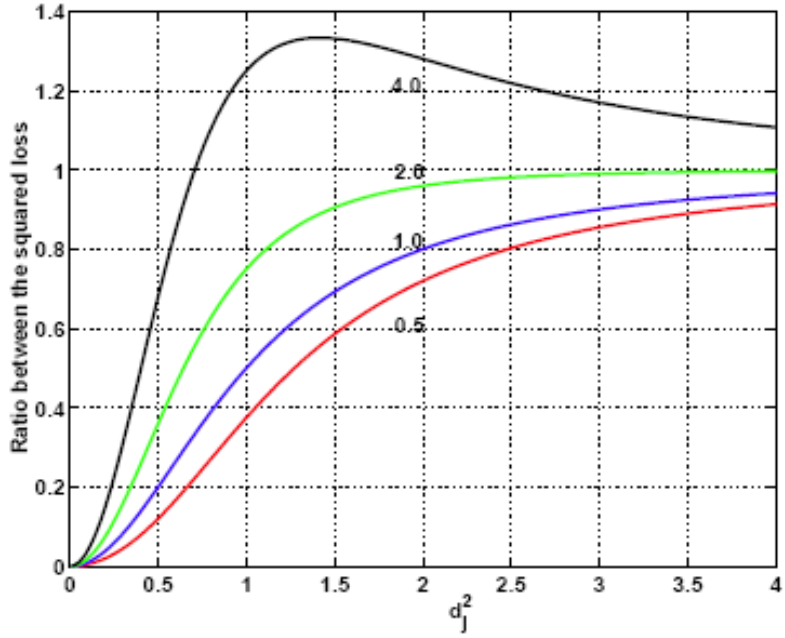


Fig 6.3

Here you can see that when β^2/σ^2 is set at 0.5, 1.0 and 2.0 the squared loss ratio is always < 1 no matter what d_j^2 is and you will always benefit by doing ridge regression (with $\lambda = 1$). If β^2/σ^2 becomes too big, at first the squared loss ratio is < 1 and you still benefit for small d_j^2 . However, as d_j^2 gets big this ratio very quickly overshoots 1 and ridge regression is not doing as well as basic linear regression. The point of this graphic is to show you that ridge regression can reduce the expected squared loss even though it uses a biased estimator.

6.3: More on Coefficient Shrinkage (Optional)

Let's illustrate why it might be beneficial in some cases to have a biased estimator. This is just an illustration with some impractical assumptions. Let's assume that $\hat{\beta}$ follows a normal distribution with mean 1 and variance 1. This means that the true $\beta = 1$ and that the true variance when you do least squares estimation is assumed to equal 1. In practice, we do not know this distribution.

Instead of $\hat{\beta}$, we will use a shrinkage estimator for β, $\tilde{\beta}$, which is $\hat{\beta}$ shrunk by a factor of *a* (where *a* is a constant greater than one). Then:

Squared loss: $E(\hat{\beta} - 1)^2 = Var(\hat{\beta})$.

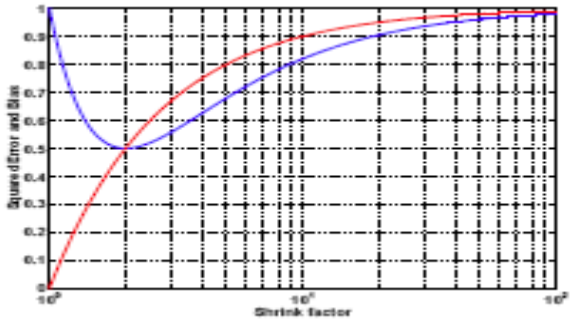
For $\tilde{\beta} = \frac{\hat{\beta}}{a}, a \geq 1, E(\tilde{\beta} - 1)^2 = Var(\tilde{\beta}) + (E(\tilde{\beta}) - 1)^2 = \frac{1}{a^2} + \left(\frac{1}{a} - 1\right)^2$.

Take a look at the squared difference between $\hat{\beta}$ and the true β (= 1). Then compare with the new estimator, $\tilde{\beta}$, and see how accurate it gets compared to the true value of 1. Again, we compute the squared difference between $\tilde{\beta}$ and 1 because $\tilde{\beta}$ itself is random and we can only talk about it in the average sense. We can think of this as a measure of accuracy - expected squared loss which turns out to be the variance of $\tilde{\beta}$ + the squared bias.

By shrinking the estimator by a factor of *a*, the bias is not zero. So, it is not an unbiased estimator anymore. The variance of $\tilde{\beta} = 1/a^2$.

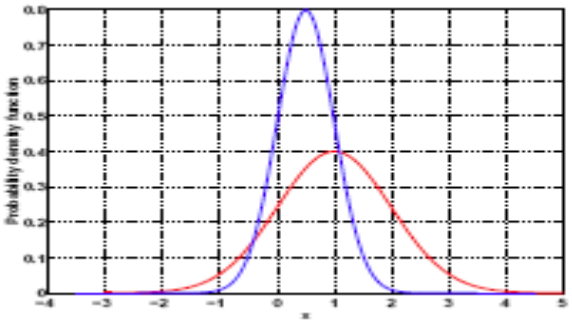
Therefore, the bigger *a* gets the higher the bias would be. The red curve in the plot below shows the squared bias with respect to *a*. When *a* goes to infinity, the bias approaches 1. Also, when *a* approaches infinity, the variance

approaches zero. As you can see, one term goes up and the other term goes down. The sum of the two terms is shown by the blue curve.



You can see that the optimal is achieved at a = 2 rather than a = 1. a = 1 gives you the unbiased estimator. However, a = 2 is biased but it gives you a smaller expected loss. In this case, a biased estimation may yield better prediction accuracy.

The red curve in the plot below represents the original distribution of β which has variance = 1. When you shrink it, dividing it by a constant greater than one, the distribution becomes spikier. The variance is decreased because the distribution is squeezed. In the meantime, there is one negative thing going on—the mean has shifted away from 1.



6.4: The Lasso

A ridge solution can be hard to interpret because it is not sparse (no β's are set exactly to 0). What if we constrain the $L1$ norm instead of the Euclidean ($L2$ norm?

$$\text{Ridge subject to: } \sum_{j=1}^{p}(\beta_j)^2 < c.$$

$$\text{Lasso subject to: } \sum_{j=1}^{p}|\beta_j| < c.$$

This is a subtle, but important change. Some of the coefficients may be shrunk exactly to zero.

The **l**east **a**bsolute **s**hrinkage and **s**election **o**perator, or *lasso*, as described in Tibshirani (1996) is a technique that has received a great deal of interest.

As with ridge regression we assume the covariates are standardized. Lasso estimates of the coefficients (Tibshirani, 1996) achieve $\min_\beta (Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j|$, so that the L2 penalty of ridge regression $\sum_{j=1}^{p} \beta_j^2$ is replaced by an L1 penalty, $\sum_{j=1}^{p} |\beta_j|$.

Let $c_0 = \sum_{j=1}^{p} |\hat{\beta}_{LS,j}|$ denote the absolute size of the least squares estimates. Values of $0 < c < c_0$ cause shrinkage towards zero.

If, for example, $c = c_0/2$ the average shrinkage of the least squares coefficients is 50%. If λ is sufficiently large, some of the coefficients are driven to zero, leading to a *sparse* model.

Geometric Interpretation

The lasso performs $L1$ shrinkage so that there are "corners'' in the constraint, which in two dimensions corresponds to a diamond. If the sum of squares"hits'' one of these corners, then the coefficient corresponding to the axis is shrunk to zero.

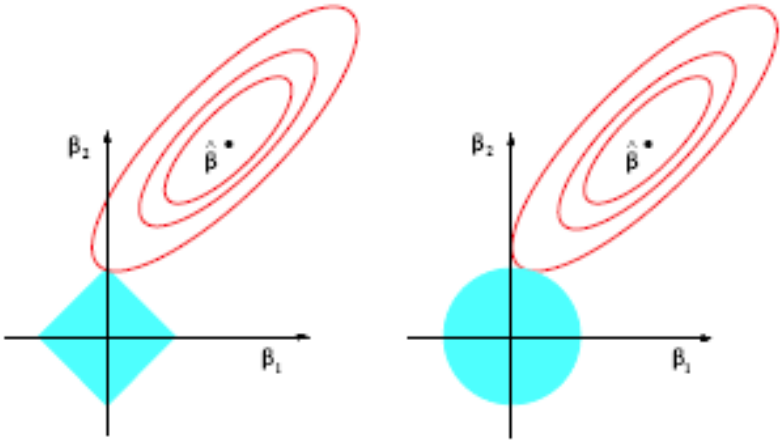


Figure 3.12: *Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions* $|\beta_1| + |\beta_2| \leq t$ *and* $\beta_1^2 + \beta_2^2 \leq t^2$, *respectively, while the red ellipses are the contours of the least squares error function.*

Fig 6.4

View the Video Explanation



Lasso

As p increases, the multidimensional diamond has an increasing number of corners, and so it is highly likely that some coefficients will be set equal to zero. Hence, the lasso performs shrinkage and (effectively) subset selection.

In contrast with subset selection, Lasso performs a *soft thresholding*: as the smoothing parameter is varied, the sample path of the estimates moves continuously to zero.

Least Angle Regression

The lasso loss function is no longer quadratic, but is still convex:

$$\text{Minimize: } \sum_{i=1}^{n}(Y_i - \sum_{j=1}^{p} X_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Unlike ridge regression, there is no analytic solution for the lasso because the solution is nonlinear in Y. The entire path of lasso estimates for all values of λ can be efficiently computed through a modification of the *Least Angle Regression* (LARS) algorithm (Efron et al. 2003).

Lasso and ridge regression both put penalties on β. More generally, penalties of the form $\lambda \sum_{j=1}^{p} |\beta_j|^q$ may be considered, for $q \geq 0$. Ridge regression and the lasso correspond to $q = 2$ and $q = 1$, respectively. When X_j is weakly related with Y, the lasso pulls β_j to zero faster than ridge regression.

Inference for Lasso Estimation

The ordinary lasso does not address the uncertainty of parameter estimation; standard errors for β's are not immediately available.

For inference using the lasso estimator, various standard error estimators have been proposed:

- Tibshirani (1996) suggested the bootstrap (Efron, 1979) for the estimation of standard errors and derived an approximate closed-form estimate.
- Fan and Li (2001) derived the sandwich formula in the likelihood setting as an estimator for the covariance of the estimates.

However, the above approximate covariance matrices give an estimated variance of 0 for predictors with $\hat{\beta}_j = 0$. The "Bayesian lasso" of Park and Casella (2008) provides valid standard errors for β and provides more stable point estimates by using the posterior median. The lasso estimate is equivalent to the mode of the posterior distribution under a normal likelihood and an independent Laplace (double exponential) prior:

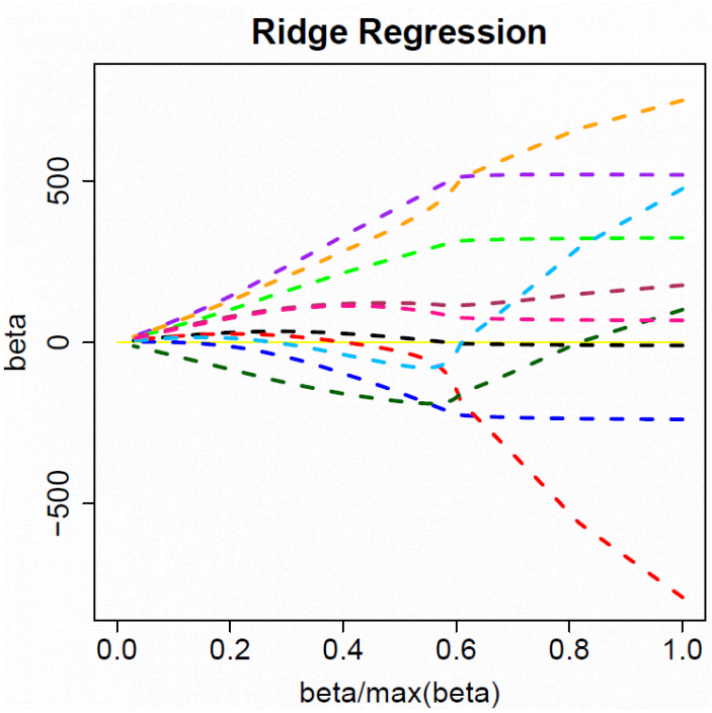
$$\pi(\beta) = \frac{\lambda}{2}\exp(-\lambda|\beta_j|)$$

The Bayesian lasso estimates (posterior medians) appear to be a compromise between the ordinary lasso and ridge regression. Park and Casella (2008) showed that the posterior density was unimodal based on a conditional Laplace prior, $\lambda|\sigma$, a noninformative marginal prior $\pi(\sigma^2) \propto 1/\sigma^2$, and the availability of a Gibbs algorithm for sampling the posterior distribution.

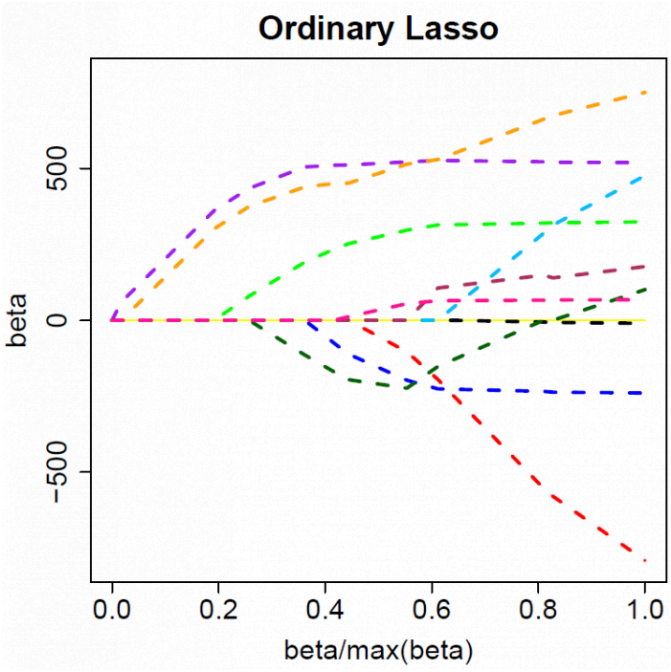
$$\pi(\beta|\sigma^2) = \prod_{j=1}^{p} \frac{\lambda}{2\sqrt{\sigma^2}}\exp(-\frac{\lambda|\beta_j|}{2\sqrt{\sigma^2}})$$

Compare Ridge Regression and Lasso

The colored lines are the paths of regression coefficients shrinking towards zero. If we draw a vertical line in the figure, it will give a set of regression coefficients corresponding to a fixed λ. (The x-axis actually shows the proportion of shrinkage instead of λ).



Ridge regression shrinks all regression coefficients towards zero; the lasso tends to give a set of zero regression coefficients and leads to a sparse solution.



Note that for both ridge regression and the lasso the regression coefficients can move from positive to negative values as they are shrunk toward zero.

Group Lasso

In some contexts, we may wish to treat a set of regressors as a group, for example, when we have a categorical covariate with more than two levels. The grouped lasso Yuan and Lin (2007) addresses this problem by considering the simultaneous shrinkage of (pre-defined) groups of coefficients.

6.5: Summary

When fitting linear shrinkage/regularization models (ridge and lasso), the predictors, X, should be standardized (for each predictor subtract the mean and then divide by the standard deviation). For a brand-new X, the prediction model is

$$\hat{y}_{new} = \hat{\beta}_0 + \sum_{j=1}^{p} \hat{\beta}_j \frac{X_{new,j} - mean(X_{train}, j)}{sd(X_{train}, j)}$$

where $\hat{\beta}$'s are estimated from the training data. $mean(X_{train}, j)$ and $sd(X_{train}, j)$ are the mean and standard deviation of the training data. The R function `1glmnet` performs this standardization by default.

Useful R functions:

- Perform ridge regression and the lasso: `glmnet` and `cv.glmnet (glmnet)`.
- (Alternative.) Fit a linear model by ridge regression: `lm.ridge (MASS).`
- (Alternative.) Fit a linear model by lasso: `lars` and `cv.lars (lars).`
- (Alternative.) Penalized regression (lasso and ridge) with cross-validation routines: (penalized).`

The Hitters example from the textbook contains specific details on using `glmnet`. You'll need to understand this in order to complete the project, which will use the diabetes data in the `lars` package.

Except where otherwise noted, content on this site is licensed under a CC BY-NC 4.0 license.

Privacy | Accessibility | Copyright
The Pennsylvania State University © 2025 |
Leave Us Feedback

OPEN.ED@PSU

Cookie Preferences



Search this site...

STAT 501 Regression Methods



13.1 - Weighted Least Squares

The method of ordinary least squares assumes that there is constant variance in the errors (which is called **homoscedasticity**). The method of **weighted least squares** can be used when the ordinary least squares assumption of constant variance in the errors is violated (which is called **heteroscedasticity**). The model under consideration is

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon^*,$$

where ϵ^* is assumed to be (multivariate) normally distributed with mean vector **0** and nonconstant variance-covariance matrix

$$\begin{pmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_n^2 \end{pmatrix}$$

If we define the reciprocal of each variance, σ_i^2, as the weight, $w_i = 1/\sigma_i^2$, then let matrix **W** be a diagonal matrix containing these weights:

$$\mathbf{W} = \begin{pmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & w_n \end{pmatrix}$$

The weighted least squares estimate is then

$$\begin{aligned}\hat{\beta}_{WLS} &= \arg\min_{\beta} \sum_{i=1}^{n} \epsilon_i^{*2} \\ &= (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{W}\mathbf{Y}\end{aligned}$$

With this setting, we can make a few observations:

- Since each weight is inversely proportional to the error variance, it reflects the information in that observation. So, an observation with a small error variance has a large weight since it contains relatively more information than an observation with a large error variance (small weight).
- The weights have to be known (or more usually estimated) up to a proportionality constant.

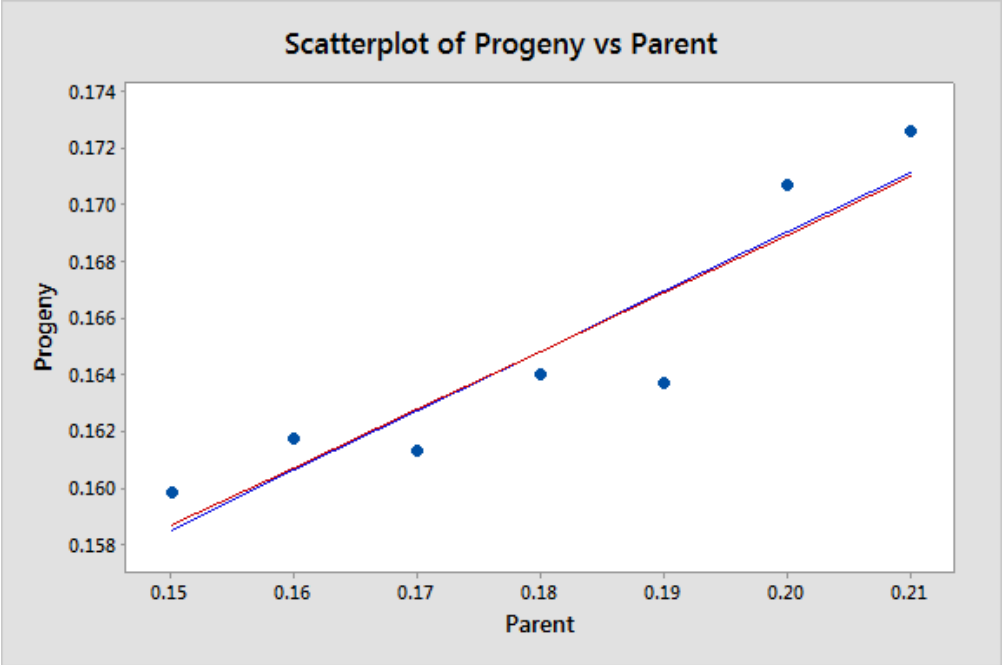
To illustrate, consider the famous 1877 Galton data set, consisting of 7 measurements each of *X* = Parent (pea diameter in inches of parent plant) and *Y* = Progeny (average pea diameter in inches of up to 10 plants grown from seeds of the parent plant). Also included in the dataset are standard deviations, SD, of the offspring peas grown from each parent. These standard deviations reflect the information in the response *Y* values (remember these are averages) and so in estimating a regression model we should downweight the observations with a large standard deviation and upweight the observations with a small standard deviation. In other words, we should use weighted least squares with weights equal to $1/SD^2$. The resulting fitted equation from Minitab for this model is:

Progeny = 0.12796 + 0.2048 Parent

Compare this with the fitted equation for the ordinary least squares model:

Progeny = 0.12703 + 0.2100 Parent

The equations aren't very different but we can gain some intuition into the effects of using weighted least squares by looking at a scatterplot of the data with the two regression lines superimposed:



The black line represents the OLS fit, while the red line represents the WLS fit. The standard deviations tend to increase as the value of Parent increases, so the weights tend to decrease as the value of Parent increases. Thus, on the left of the graph where the observations are up-weighted the red fitted line is pulled slightly closer to the data points, whereas on the right of the graph where the observations are down-weighted the red fitted line is slightly further from the data points.

For this example, the weights were known. There are other circumstances where the weights are known:

- If the *i*-th response is an average of n_i equally variable observations, then $Var\left(y_i\right) = \sigma^2/n_i$ and $w_i = n_i$.
- If the *i*-th response is a total of n_i observations, then $Var\left(y_i\right) = n_i\sigma^2$ and $w_i = 1/ n_i$.
- If variance is proportional to some predictor x_i, then $Var\left(y_i\right) = x_i\sigma^2$ and $w_i = 1/ x_i$.

In practice, for other types of datasets, the structure of **W** is usually unknown, so we have to perform an ordinary least squares (OLS) regression first. Provided the regression function is appropriate, the i-th squared residual from the OLS fit is an estimate of σ_i^2 and the i-th absolute residual is an estimate of σ_i (which tends to be a more useful estimator in the presence of outliers). The residuals are much too variable to be used directly in estimating the weights, w_i, so instead we use either

the squared residuals to estimate a variance function or the absolute residuals to estimate a standard deviation function. We then use this variance or standard deviation function to estimate the weights.

Some possible variance and standard deviation function estimates include:

- If a residual plot against a predictor exhibits a megaphone shape, then regress the absolute values of the residuals against that predictor. The resulting fitted values of this regression are estimates of σ_i. (And remember $w_i = 1/\sigma_i^2$).
- If a residual plot against the fitted values exhibits a megaphone shape, then regress the absolute values of the residuals against the fitted values. The resulting fitted values of this regression are estimates of σ_i.
- If a residual plot of the squared residuals against a predictor exhibits an upward trend, then regress the squared residuals against that predictor. The resulting fitted values of this regression are estimates of σ_i^2.
- If a residual plot of the squared residuals against the fitted values exhibits an upward trend, then regress the squared residuals against the fitted values. The resulting fitted values of this regression are estimates of σ_i^2.

After using one of these methods to estimate the weights, w_i, we then use these weights in estimating a weighted least squares regression model. We consider some examples of this approach in the next section.

Some key points regarding **weighted least squares** are:

1. The difficulty, in practice, is determining estimates of the error variances (or standard deviations).
2. Weighted least squares estimates of the coefficients will usually be nearly the same as the "ordinary" unweighted estimates. In cases where they differ substantially, the procedure can be iterated until estimated coefficients stabilize (often in no more than one or two iterations); this is called *iteratively reweighted least squares*.
3. In some cases, the values of the weights may be based on theory or prior research.

4. In designed experiments with large numbers of replicates, weights can be estimated directly from sample variances of the response variable at each combination of predictor variables.
5. The use of weights will (legitimately) impact the widths of statistical intervals.

« Previous Next »

Lesson

Lesson 1: Simple Linear Regression
Lesson 2: SLR Model Evaluation
Lesson 3: SLR Estimation & Prediction
Lesson 4: SLR Model Assumptions
Lesson 5: Multiple Linear Regression
Lesson 6: MLR Model Evaluation
Lesson 7: MLR Estimation, Prediction & Model Assumptions
Lesson 8: Categorical Predictors
Lesson 9: Data Transformations
Lesson 10: Model Building
Lesson 11: Influential Points
Lesson 12: Multicollinearity & Other Regression Pitfalls
Lesson 13: Weighted Least Squares & Logistic Regressions
13.1 - Weighted Least Squares
13.1.1 - Weighted Least Squares Examples
13.2 - Logistic Regression

Software Help 13

Optional Content

Topic 1: Robust Regression

Topic 2: Time Series & Autocorrelation

Topic 3: Poisson & Nonlinear Regression

Resource Menu

Notation Used in this Course

Common Procedures in Minitab

Search this site...

STAT 501 Regression Methods

13.1.1 - Weighted Least Squares Examples

Example 13-1: Computer-Assisted Learning Dataset

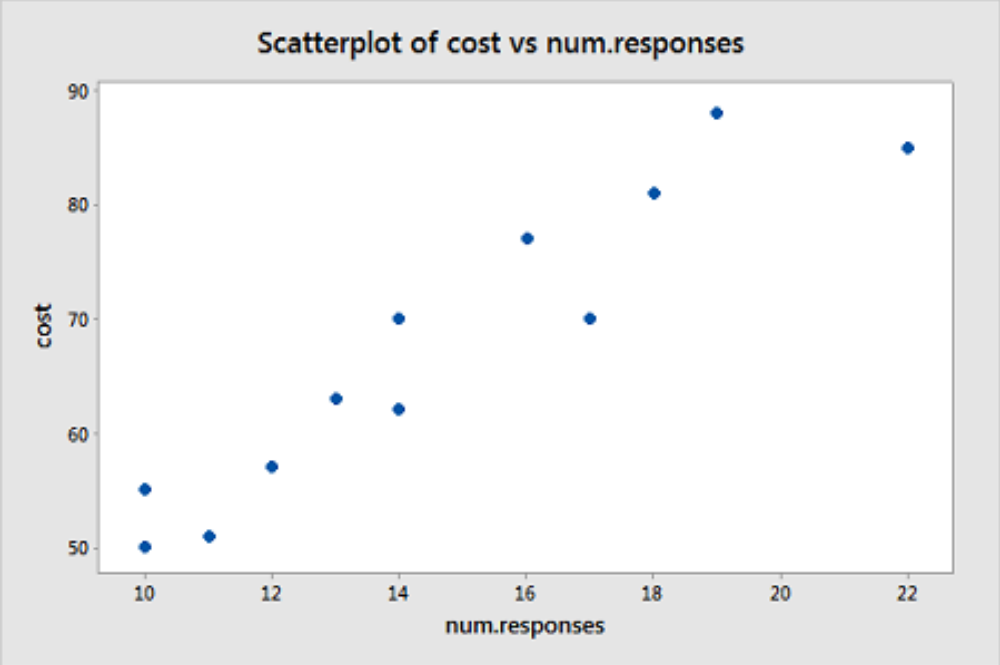


The Computer-Assisted Learning New data was collected from a study of computer-assisted learning by n = 12 students.

i	Responses	Cost
1	16	77
2	14	70
3	22	85
4	10	50
5	14	62
6	17	70
7	10	55
8	13	63
9	19	88
10	12	57
11	18	81

i	Responses	Cost
12	11	51

The response is the cost of the computer time (*Y*) and the predictor is the total number of responses in completing a lesson (*X*). A scatterplot of the data is given below.



From this scatterplot, a simple linear regression seems appropriate for explaining this relationship.

First, an ordinary least squares line is fit to this data. Below is the summary of the simple linear regression fit for this data

Model Summary

S	R-sq	R-sq(adj)	R-sq(pred)
4.59830	88.91%	87.80%	81.27%

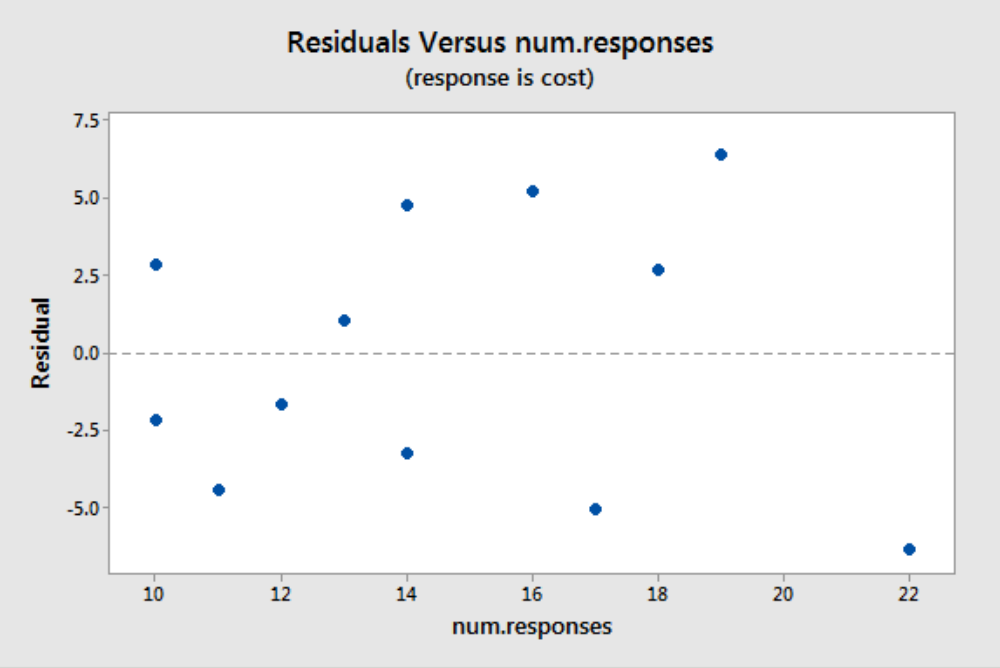
Coefficients

Term	Coef	SE Coef	T-Value	P-Value	VIF
Constant	19.47	5.52	3.53	0.005	
num.responses	3.269	0.365	8.95	0.000	1.00

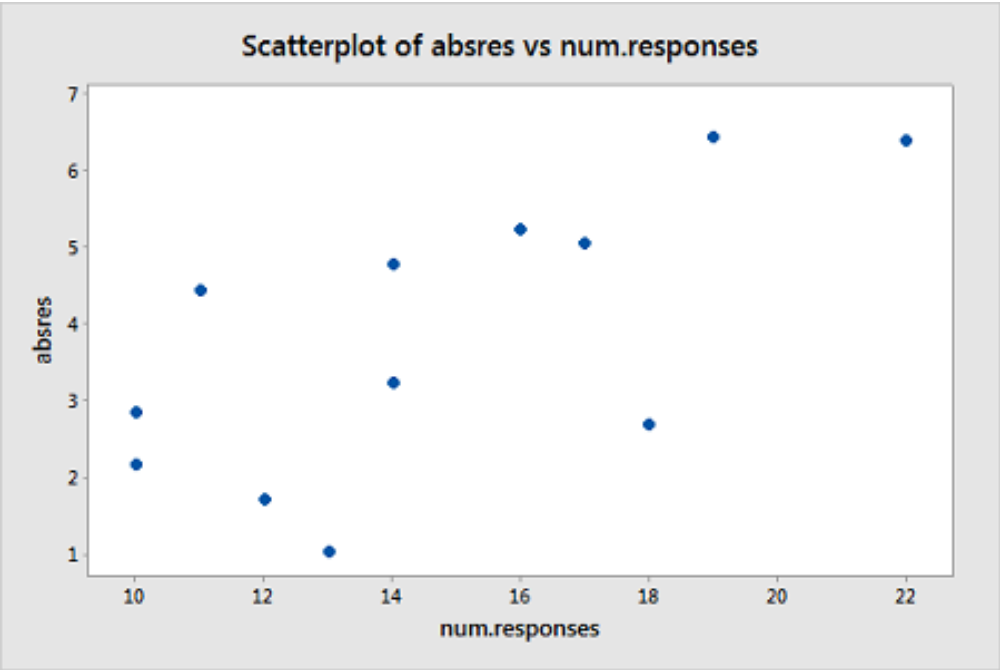
Regression Equation

cost = 19.47 + 3.269 num.responses

A plot of the residuals versus the predictor values indicates possible nonconstant variance since there is a very slight "megaphone" pattern:



We will turn to weighted least squares to address this possibility. The weights we will use will be based on regressing the absolute residuals versus the predictor. In Minitab, we can use the Storage button in the Regression Dialog to store the residuals. Then we can use Calc > Calculator to calculate the absolute residuals. A plot of the absolute residuals versus the predictor values is as follows:



The weights we will use will be based on regressing the absolute residuals versus the predictor. Specifically, we will fit this model, use the Storage button to store the fitted values, and then use Calc > Calculator to define the weights as 1 over the squared fitted values. Then we fit a weighted least squares regression model by fitting a linear regression model in the usual way but clicking "Options" in the Regression Dialog and selecting the just-created weights as "Weights."

The summary of this weighted least squares fit is as follows:

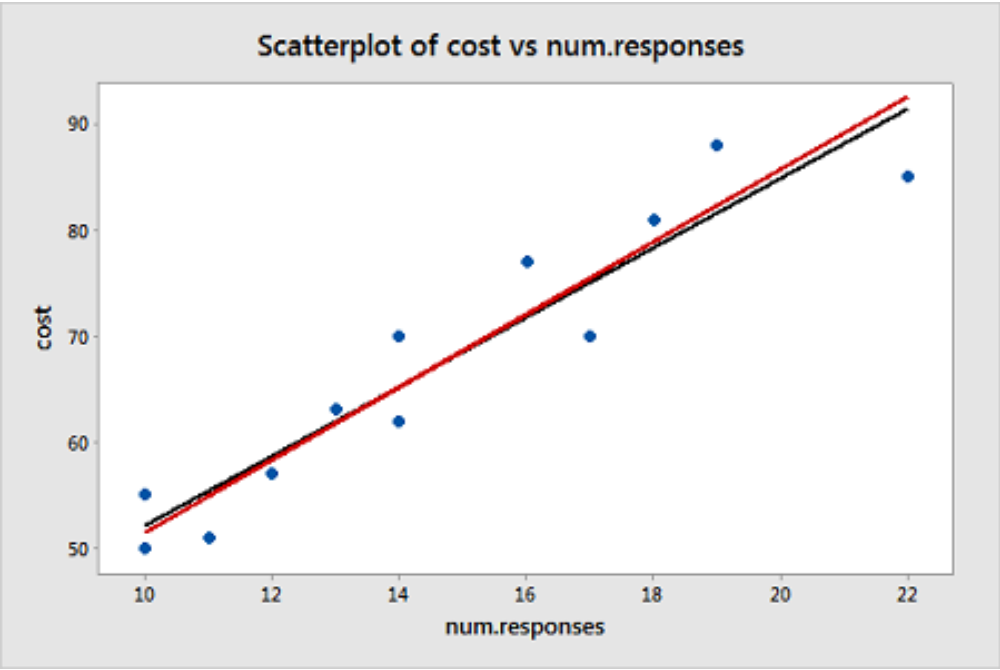
Model Summary

S	R-sq	R-sq(adj)	R-sq(pred)
1.15935	89.51%	88.46%	83.87%

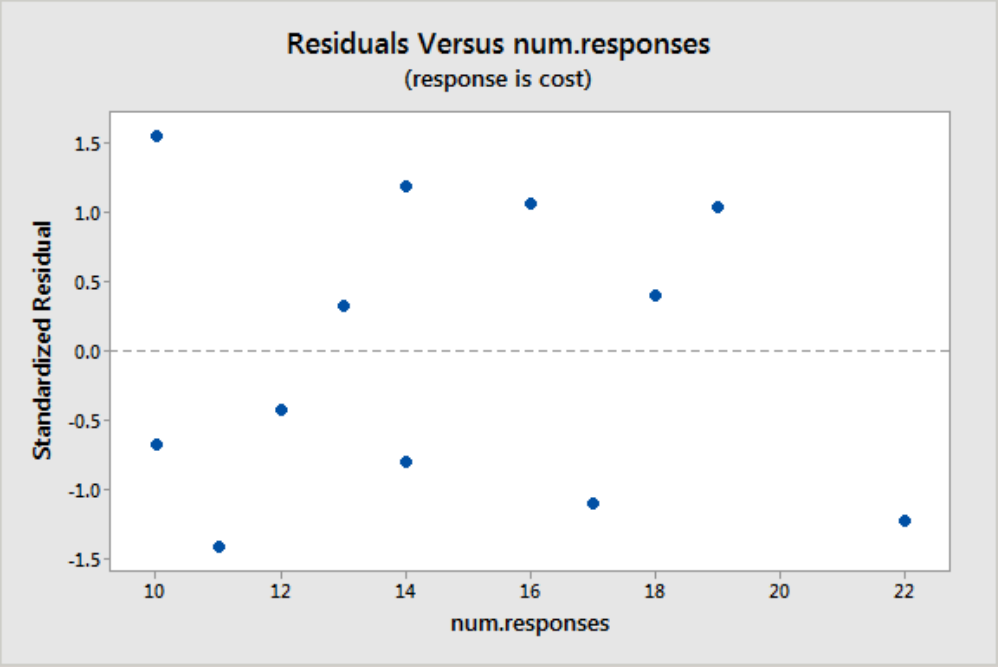
Coefficients

Term	Coef	SE Coef	T-Value	P-Value	VIF
Constant	17.30	4.83	3.58	0.005	
num.responses	3.421	0.370	9.24	0.000	1.00

Notice that the regression estimates have not changed much from the ordinary least squares method. The following plot shows both the OLS fitted line (black) and WLS fitted line (red) overlaid on the same scatterplot.



A plot of the studentized residuals (remember Minitab calls these "standardized" residuals) versus the predictor values when using the weighted least squares method shows how we have corrected for the megaphone shape since the studentized residuals appear to be more randomly scattered about 0:



With weighted least squares, it is crucial that we use studentized residuals to evaluate the aptness of the model since these take into account the weights that are used to model the changing variance. The usual residuals don't do this and will maintain the same non-constant variance pattern no matter what weights have been used in the analysis.

Example 13-2: Market Share Data

Here we have market share data for n = 36 consecutive months (Market Share data). Let Y = market share of the product; X_1 = price; X_2 = 1 if the discount promotion is in effect and 0 otherwise; X_2X_3 = 1 if both discount and package promotions in effect and 0 otherwise. The regression results below are for a useful model in this situation:

Coefficients

Term	Coef	SE Coef	T-Value	P-Value	VIF
Constant	3.196	0.356	8.97	0.000	
Price	-0.334	0.152	-2.19	0.036	1.01

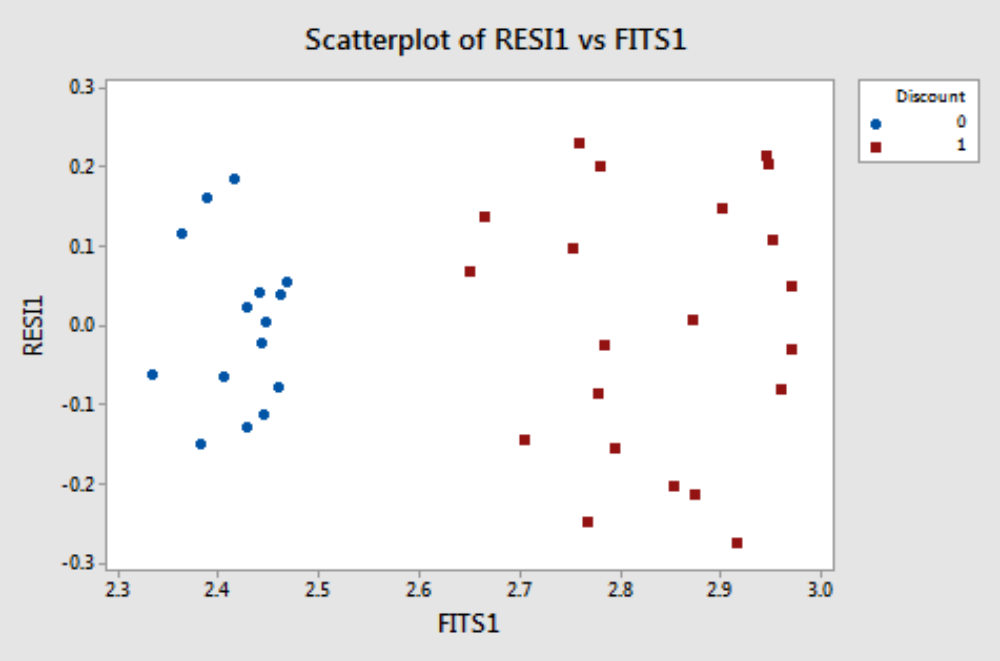
Term	Coef	SE Coef	T-Value	P-Value	VIF
Discount	0.3081	0.0641	4.80	0.000	1.68
DiscountPromotion	0.1762	0.0660	2.67	0.012	1.69

This model represents three different scenarios:

1. Months in which there was no discount (and either a package promotion or not): X2 = 0 (and X3 = 0 or 1);
2. Months in which there was a discount but no package promotion: X2 = 1 and X3 = 0;
3. Months in which there was both a discount and a package promotion: X2 = 1 and X3 = 1.

So, it is fine for this model to break the hierarchy if there is no significant difference between the months in which there was no discount and no package promotion and months in which there was no discount but there was a package promotion.

A residual plot suggests nonconstant variance related to the value of X_2:



From this plot, it is apparent that the values coded as 0 have a smaller variance than the values coded as 1. The residual variances for the two separate groups defined by the discount pricing variable are:

Discount	N	StDev	Variance	95% CI for StDevs
0	15	0.103	0.011	(0.077, 0.158)
1	21	0.164	0.027	(0.136, 0.217)

Because of this nonconstant variance, we will perform a weighted least squares analysis. For the weights, we use $w_i = 1/\hat{\sigma}_i^2$ for *i* = 1, 2 (in Minitab use Calc > Calculator and define "weight" as 'Discount'/0.027 + (1-'Discount')/0.011 . The weighted least squares analysis (set the just-defined "weight" variable as "weights" under Options in the Regression dialog) is as follows:

Analysis of Variance

Source	DF	Adj SS	Adj MS	F-Value	P-Value
Regression	3	96.109	32.036	30.84	0.000
Price	1	4.688	4.688	4.51	0.041
Discount	1	23.039	23.039	22.18	0.000
DiscountPromotion	1	5.634	5.634	5.42	0.026
Error	32	33.246	1.039		
Total	35	129.354			

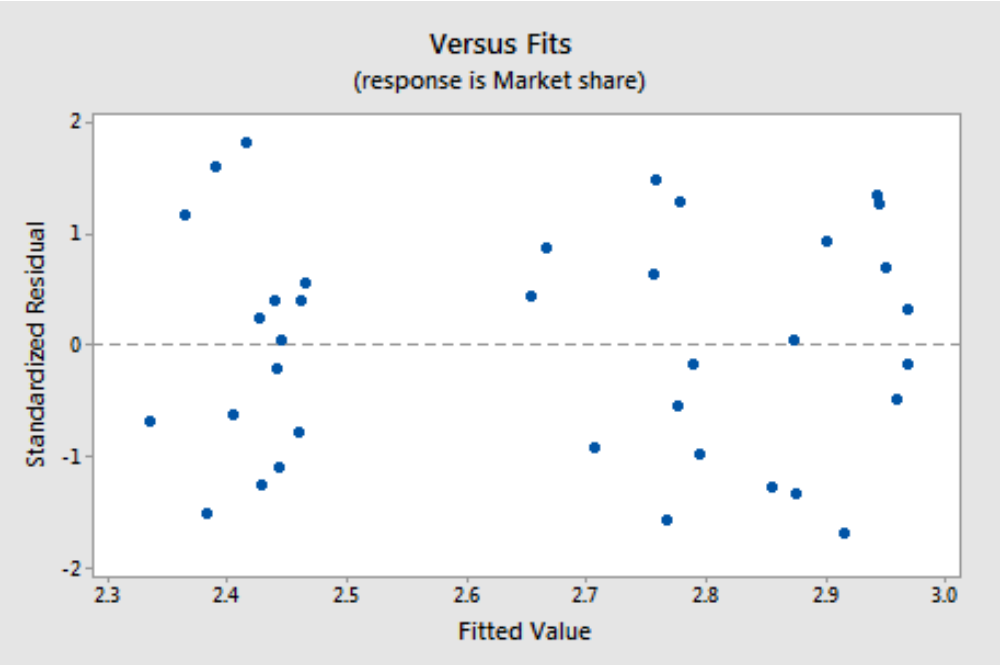
Coefficients

Term	Coef	SE Coef	T-Value	P-Value	VIF
Constant	3.175	0.357	8.90	0.000	
Price	-0.325	0.153	-2.12	0.041	1.01
Discount	0.3083	0.0655	4.71	0.000	2.04
DiscountPromotion	0.1759	0.0755	2.33	0.026	2.05

An important note is that Minitab's ANOVA will be in terms of the weighted SS. When doing a weighted least squares analysis, you should note how different the SS values of the weighted case are from the SS values for the unweighted case.

Also, note how the regression coefficients of the weighted case are not much different from those in the unweighted case. Thus, there may not be much of an obvious benefit to using the weighted analysis (although intervals are going to be more reflective of the data).

If you proceed with a weighted least squares analysis, you should check a plot of the residuals again. Remember to use the studentized residuals when doing so! For this example, the plot of studentized residuals after doing a weighted least squares analysis is given below and the residuals look okay (remember Minitab calls these standardized residuals).



Example 13-3: Home Price Dataset

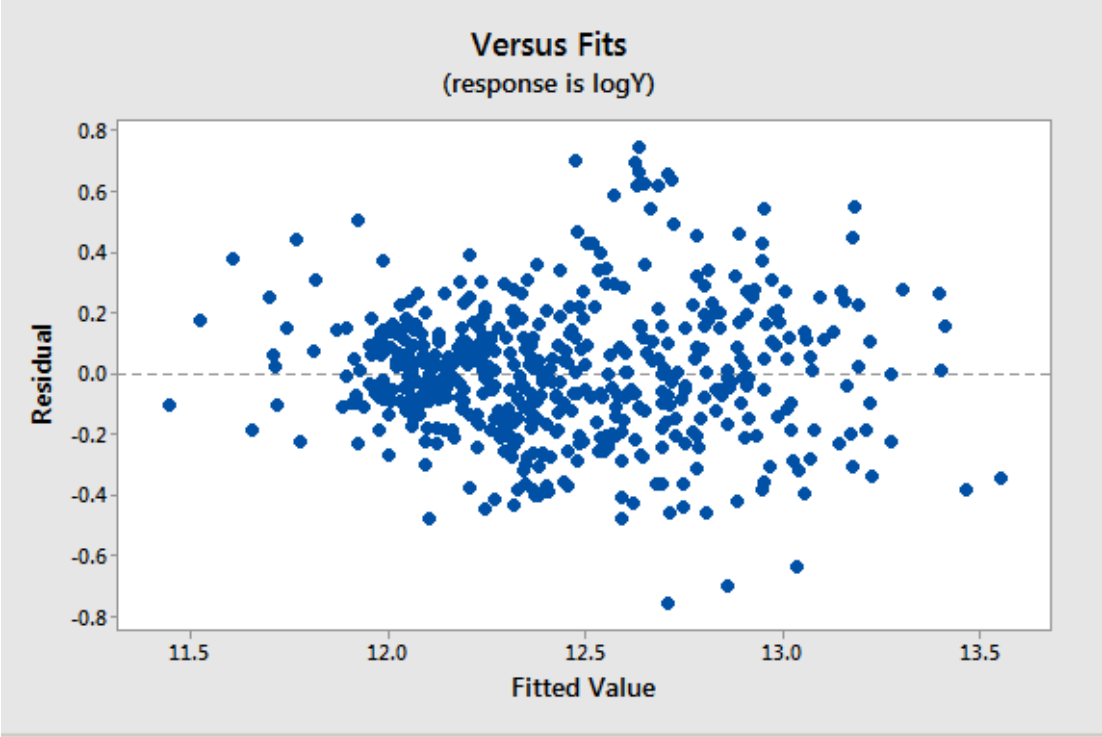
The Home Price data set has the following variables:



Y = sale price of a home
X_1 = square footage of the home
X_2 = square footage of the lot

Since all the variables are highly skewed we first transform each variable to its natural logarithm. Then when we perform a regression analysis and look at a plot of the residuals versus the fitted values (see below), we note a slight “megaphone” or “conic” shape of the residuals.

Term	Coef	SE Coeff	T-Value	P-Value	VIF
Constant	1.964	0.313	6.28	0.000	
logX1	1.2198	0.0340	35.87	0.000	1.05
logX2	0.1103	0.0241	4.57	0.000	1.05



We interpret this plot as having a mild pattern of nonconstant variance in which the amount of variation is related to the size of the mean (which are the fits).

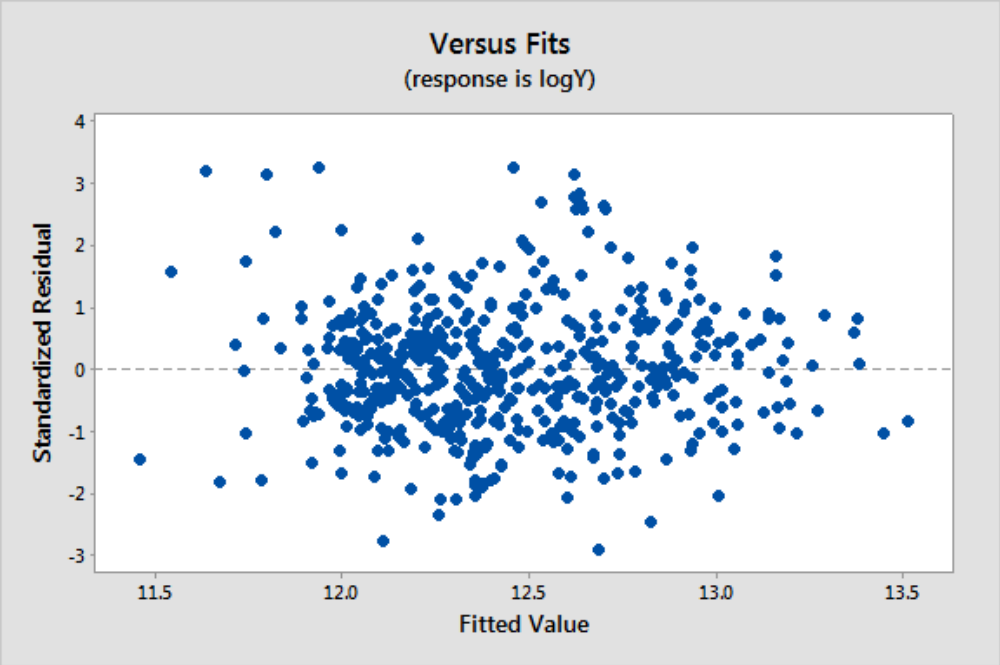
So, we use the following procedure to determine appropriate weights:

- Store the residuals and the fitted values from the ordinary least squares (OLS) regression.
- Calculate the absolute values of the OLS residuals.
- Regress the absolute values of the OLS residuals versus the OLS fitted values and store the fitted values from this regression. These fitted values are estimates of the error standard deviations.
- Calculate weights equal to $1/fits^2$, where "fits" are the fitted values from the regression in the last step.

We then refit the original regression model but use these weights this time in a weighted least squares (WLS) regression.

Results and a residual plot for this WLS model:

Term	Coef	SE Coeff	T-Value	P-Value	VIF
Constant	2.377	0.284	8.38	0.000	
logX1	1.2014	0.0336	35.72	0.000	1.08
logX2	0.0831	0.0217	3.83	0.000	1.08



« Previous Next »

Lesson

Lesson 1: Simple Linear Regression
Lesson 2: SLR Model Evaluation
Lesson 3: SLR Estimation & Prediction
Lesson 4: SLR Model Assumptions
Lesson 5: Multiple Linear Regression
Lesson 6: MLR Model Evaluation
Lesson 7: MLR Estimation, Prediction & Model Assumptions
Lesson 8: Categorical Predictors

Lesson 9: Data Transformations

Lesson 10: Model Building

Lesson 11: Influential Points

Lesson 12: Multicollinearity & Other Regression Pitfalls

Lesson 13: Weighted Least Squares & Logistic Regressions

13.1 - Weighted Least Squares

13.1.1 - Weighted Least Squares Examples

13.2 - Logistic Regression

Software Help 13

Optional Content

Topic 1: Robust Regression

Topic 2: Time Series & Autocorrelation

Topic 3: Poisson & Nonlinear Regression

Resource Menu

Notation Used in this Course

Common Procedures in Minitab

FINANCIAL ECONOMETRICS

MODULE 2 | LESSON 2

ROBUST REGRESSION

Reading Time	90 minutes
Prior Knowledge	OLS regression, Weighted Least Square Regression, Linear Algebra
Keywords	Objective Function, Robust Regression, Quadratic Function, M-Estimation, Huber Method, Bisquare Method, Iterative Reweighted Least Square Regression (IRLS)

In the last lesson, we discussed how to use weights to handle unequal error terms from observations (heteroskedasticity) in a regression model. We also briefly mentioned that using weights can deal with outlier issues as well. In this lesson, we will dive deeper into how outliers can influence OLS regression model results. We will then propose an adjusted regression method to handle outliers: robust regression. There are several methods for robust regression. We will focus on two types: M-estimation and L1 regression.

In [1]:
```
# Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.graphics.regressionplots import abline_plot

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
```

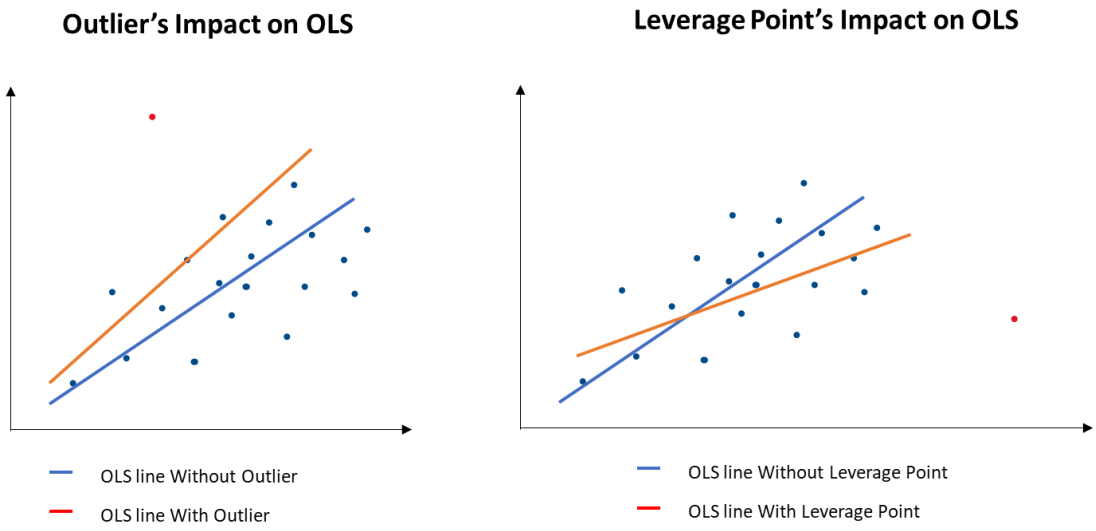
In [2]:
```
# Import data
data_set = pd.read_csv("M2. module_2_data.csv")
```

1. Problems of Extreme Points for Ordinary Least Square (OLS) Regression

In many financial problems, it is common to have extreme points in the dataset for analysis. These can come from measurement error by the person who records the measurements, incorrect data input, or simply an outlier. Not all extreme points are bad data points. In asset return analysis, abnormal returns show up every now and again. These abnormal returns are actually an interesting topic for a lot of researchers. Assume that, after cleaning up the dataset and removing all the bad data points, you still find extreme points in your dataset. What should you do?

In Module 1, Lesson 1, we discussed the impact of extreme points (**outliers** and **leverage points**) on the estimation of an OLS (ordinary least square) regression model. We know that not all extreme points have significant impacts on the OLS regression results. The extreme points that we are worried about are **influential points**, which usually change OLS regression results. Let's first use graphs to demonstrate this issue.

Figure 1: Impact of Outliers and Leverage Points on OLS Regression



From Figure 1, we can see an outlier in the dataset; this outlier will pull the regression line closer to the outlier because the outlier has the largest distance from the regression line compared to other data points. We know from the required reading from Module 1, Lesson 1 that OLS regression minimizes the squared sum of the residuals (the distance between the data points and the regression line) to obtain coefficient estimates. Since the distance between the outlier and the regression line is the largest, the minimization mechanism will naturally pull the line closer to the outlier in order to get the smallest sum of all distances from data points to the line.

So what can we do with outliers? Remember from the last lesson how we used weight to adjust each data point's influence in weighted least square (WLS) regression. Although WLS was used to tackle unequal variance in error terms, WLS can also give less weight to a data point further away from the rest of the data points. Hence, WLS is one option.

When there are only a few extreme points in the dataset, we can use a scatter plot or a residual plot to detect the extreme points. However, when there are many extreme points in the dataset, it is difficult to manually remove or even detect them. We need a method that can address extreme points in the model. The weighted least square regression model shown in the last lesson is one of the methods for handling extreme points. Robust regression is another method that we will introduce in this lesson.

2. Robust Regression

In the last lesson, we learned that a weighted least square regression model applies weight to its objective function in order to reduce the impact of extreme points. In robust

regression, different objective functions are applied to reduce the impact of extreme points. There are several methods for choosing an objective function using robust regression. In this lesson, we will focus on M-estimation and will briefly talk about other methods later on in this lesson.

2.1 M-Estimation

2.1.1 Basics of M-Estimation

We introduced the concept of objective function in the last lesson. An **objective function** is a function we would like to maximize or minimize. In a simple OLS regression, we minimize the following objective function:

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (Y_i - \beta_0 - \beta_1 X_i)^2$$

with respect to the coefficients β_0 and β_1 and compute $\hat{\beta}_0$ and $\hat{\beta}_1$ by solving the two equations associated with the first order conditions.

According to the objective function above, we want to minimize the sum of squared residuals for OLS regression. We already see that using squared residuals can put more emphasis on outliers when minimizing the objective function. A squared function is a **quadratic function**. When $f(x) = x^2,\ f(x)$ is the quadratic function of x. Since a quadratic function of residuals from OLS puts a lot of emphasis on outliers, *M-estimation* proposes other function forms to avoid this issue.

M-Estimation proposes the following objective function:

$$\sum_{i=1}^{n} \rho(e_i) = \sum_{i=1}^{n} \rho(Y_i - X_i^t \beta)$$

We can see ρ is the contribution of each residual to the *M-estimation* objective function. $\rho(\cdot)$ should have the following properties:

1. $\rho(e) \geq 0$ i.e. ρ cannot be negative
2. $\rho(0) = 0$ i.e. When the argument of ρ is 0, ρ is also 0
3. $\rho(e) = \rho(-e)$ i.e. ρ is symmetric
4. When $|e_i| > |e_j|$, then $\rho(e_i) > \rho(e_j)$ i.e. ρ is monotone

Let's take a quick look of some ρ examples. Two of the popular ρ functions are $\rho(e) = e^2$ and $\rho(e) = |e|$. We will talk more about them later in this lesson.

We then take the partial derivatives of this objective function with respect to β and set them to 0. We will get the following equations:

$$\sum_{i=1}^{n} \frac{\partial \rho}{\partial e} X_i^t = 0$$

Let's assign $\psi(e) = \frac{\partial \rho}{\partial e}$. ψ is called an influence function.

We can rewrite the above equations as follows:

$$\sum_{i=1}^{n} \psi(e_i) X_i^t = 0$$

Let's define a weight function $w(e) = \frac{\psi(e)}{e}$ and let $w(e_i) = w_i$.

We can rewrite the above function again as follows:

$$\sum_{i=1}^{n} w_i e_i X_i^t = 0$$

Why did we do all these steps? If you look at the above equation, it is the same partial derivative equation from the WLS lesson. We have transformed the *M-estimation* to weighted least square regression.

Now, we have to calculate the estimates of *M-estimation*. From the above description, it is the same as solving a WLS problem. From our weight formula, we know the weights depend on residuals. The residuals depend on estimated coefficients. The coefficients depend on weights. It is a circle. We usually would start from estimating the coefficients using OLS. Then, we'd get the residuals and estimate the weights, and we would estimate weighted least square regression and repeat the whole process again. This is called **iterative reweighted least squares regression (IRLS).**

Here are the steps of **iterative reweighted least squares** regression:

1. Use OLS to estimate initial coefficients.
2. Obtain the residuals of the OLS model and generate weights.
3. Estimate coefficients using WLS and the weights from Step 2.
4. Plug coefficients from WLS into the linear model and calculate new residuals and repeat from Step 2 to Step 4 until the coefficients converge.

2.1.2 $\rho()$ Functions: Huber Method and Bisquare Method

After setting out the steps to run an M-estimation using IRLS in the last section, we now need to find weights. How should we find weights? From the last section, we know weight functions can be derived from ρ functions. In this section, we are going to introduce two commonly used ρ functions: the Huber method and bisquare method. We will use them to derive weight functions.

The ρ function for the **Huber method** is as follows:

$$\rho_{Huber}(e) = \begin{cases} \frac{1}{2}e^2, & \text{if } |e| \leq k \\ k|e| - \frac{1}{2}k^2, & \text{if } |e| > k \end{cases}$$

k is called a **tuning constant.** A smaller k will make the objective function more resistant to outliers. We usually set $k = 1.345\sigma$. σ is the standard deviation of the errors. Why 1.345? If the errors are normally distributed, we will get 95% efficiency when we estimate

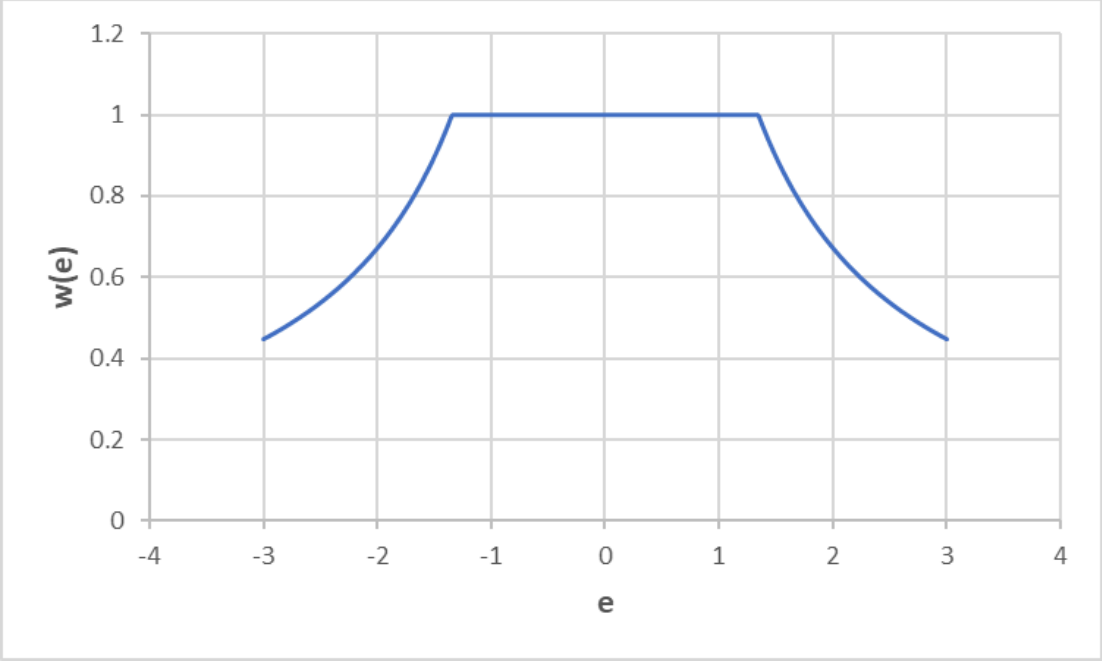
the errors using residuals. This level of efficiency is similar to that used for OLS regression (95%).

One more point: We usually use standard deviation of residuals to estimate σ. However, one outlier can make the estimated value of the standard deviation of residuals very sensitive because the mean is part of the calculation of standard deviation. Mean is sensitive to outliers. Instead, we use **median absolute deviation (MAD)** = $\text{median}|\text{residuals}|$ to estimate the standard deviation of errors σ.

Now that we know the ρ function for the Huber method, we can derive the weight function. The weight function for the Huber method is as follows:

$$w_{Huber}(e) = \begin{cases} 1, & \text{if } |e| \leq k \\ \frac{k}{|e|}, & \text{if } |e| > k \end{cases}$$

Figure 2: Huber Method Weight Function Chart



From Figure 2, we can see that the weights for data points are the same when their errors are within (-1.345, 1.345). Once the error passes the band, the value of the weight starts to drop quickly, which will give less weight to outliers.

The ρ function for the **bisquare method** is as follows:

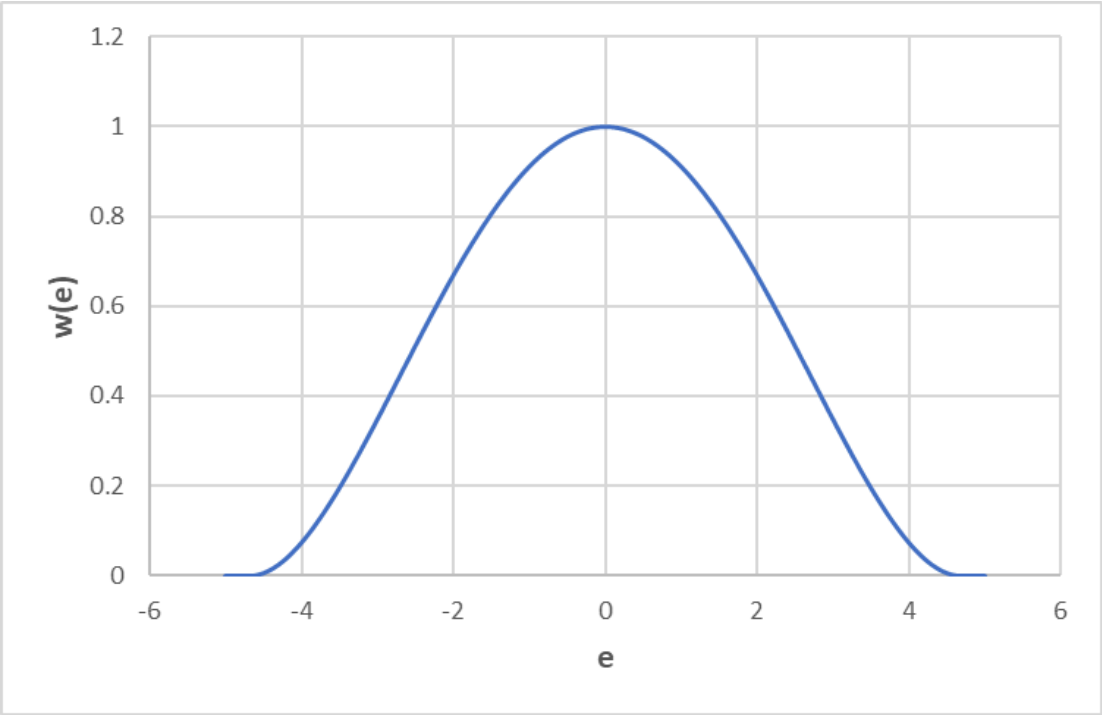
$$\rho_{Bisquare} = \begin{cases} \frac{k^2}{6}(1-(1-(\frac{e}{k})^2)^3), & \text{if } |e| \leq k \\ \frac{k^2}{6}, & \text{if } |e| > k \end{cases}$$

The concept of k here is the same as the k in the Huber method. For the bisquare method, $k = 4.685\sigma$. It also uses MAD to estimate standard deviation of error.

The weight function for the bisquare method is as follows:

$$w_{Bisquare}(e) = \begin{cases} (1-(\frac{e}{k})^2)^2, & \text{if } |e| \leq k \\ 0, & \text{if } |e| > k \end{cases}$$

Figure 3: Bisquare Method Weight Function Chart



We can see from Figure 3 that the bisquare method de-emphasizes outliers faster than the Huber method. The bisquare method starts to decrease the weight when error begins to show up.

The residual sum of a square for OLS is also a ρ function form.

$$\rho(e) = e^2$$

This function also meets all of the properties of an ρ function.

The weight function for OLS is:

$$w(e) = 1$$

From the weight function, we can see OLS gives all data points a weight of 1 regardless of their outlier status.

2.1.3 Pros and Cons of M-Estimators

Both the Huber method and bisquare method are better regression methods than classic OLS regression because the Huber method and bisquare method assign less weight to outliers.

One thing that is very helpful in robust regression is outlier detection. After running a robust regression model, you can check the weight associated with each data point to identify outliers.

However, the bisquare method is popular because it starts to de-emphasize a data point the moment the error shows up. However, a bisquare result is sensitive to the choice of initial coefficient estimates. One way to get away from this issue is to use the Huber method at the beginning to get the coefficients. Then, use the Huber method coefficients in the bisquare method.

Another issue with the Huber method and the bisquare method is that they still cannot handle leverage points (points with large X values) well. There is another branch of robust regression to handle this issue: bounded influence regression. Some examples of this method include least trimmed squares, least median squares, etc.

3. Foreign Exchange Market Analysis Using Robust Regression

We will continue our example from the last lesson. In the last lesson, we used WLS regression to understand how U.S. 10-year treasury bond yield would influence the U.S. Dollar Index. In this lesson, we are going run robust regression on the same example. We will use the Huber method and bisquare method respectively and compare the results. I will also reproduce the WLS result here to compare with the Huber method and bisquare method.

Figure 4: Huber Method - U.S. Dollar Index Daily Return On 10-Year Treasury Bond Yield

In [3]:
```
# Robust Regression model
rr_huber = sm.RLM(
    data_set["DXY"], sm.add_constant(data_set["X10Y_TBY"]), M=sm.robust.norms.Hu
)
rr_huber_results = rr_huber.fit()
rr_huber_results.summary()
```

Out[3]:

Robust linear Model Regression Results

Dep. Variable:	DXY	**No. Observations:**	250
Model:	RLM	**Df Residuals:**	248
Method:	IRLS	**Df Model:**	1
Norm:	HuberT		
Scale Est.:	mad		
Cov Type:	H1		
Date:	Fri, 05 Dec 2025		
Time:	19:01:32		
No. Iterations:	19		

	coef	std err	z	P>|z|	[0.025	0.975]
const	9.673e-05	0.000	0.367	0.714	-0.000	0.001
X10Y_TBY	0.0446	0.011	4.152	0.000	0.024	0.066

If the model instance has been used for another fit with different fit parameters, then the fit options might not be the correct ones anymore .

Figure 5: Bisquare Method - U.S. Dollar Index Daily Return On 10-Year Treasury Bond Yield

In [4]:

```
# Bisquare Method model
rr_bisquare = sm.RLM(
    data_set["DXY"],
    sm.add_constant(data_set["X10Y_TBY"]),
    M=sm.robust.norms.TukeyBiweight(),
)
rr_bisquare_results = rr_bisquare.fit()
rr_bisquare_results.summary()
```

Out[4]:

Robust linear Model Regression Results

Dep. Variable:	DXY	**No. Observations:**	250
Model:	RLM	**Df Residuals:**	248
Method:	IRLS	**Df Model:**	1
Norm:	TukeyBiweight		
Scale Est.:	mad		
Cov Type:	H1		
Date:	Fri, 05 Dec 2025		
Time:	19:01:32		
No. Iterations:	18		

	coef	std err	z	P>|z|	[0.025	0.975]
const	6.698e-05	0.000	0.254	0.800	-0.000	0.001
X10Y_TBY	0.0515	0.011	4.797	0.000	0.030	0.073

If the model instance has been used for another fit with different fit parameters, then the fit options might not be the correct ones anymore .

Figure 6: WLS - U.S. Dollar Index Daily Return On 10 Year Treasury Bond Yield

In [5]:

```
# WLS model
model_1 = smf.ols("DXY ~ X10Y_TBY", data=data_set).fit()

# Add Absolute residuals and fitted values to dataset columns
data_set["abs_residuals"] = np.abs(model_1.resid)
data_set["fitted_values"] = model_1.fittedvalues

# Fit OLS model with absolute residuals and fitted values
model_temp = smf.ols("abs_residuals ~ fitted_values", data=data_set).fit()

# Compute weights and add it to the data_set column
weights = model_temp.fittedvalues
weights = weights**-2
data_set["weights"] = weights

# WLS model
Y = data_set["DXY"].tolist()
X = data_set["X10Y_TBY"].tolist()
X = sm.add_constant(X)  # add a intercept point

model_WLS = sm.WLS(Y, X, data_set["weights"]).fit()
model_WLS.summary()
```

Out[5]:

WLS Regression Results

Dep. Variable:	y	**R-squared:**	0.100
Model:	WLS	**Adj. R-squared:**	0.096
Method:	Least Squares	**F-statistic:**	27.59
Date:	Fri, 05 Dec 2025	**Prob (F-statistic):**	3.23e-07
Time:	19:01:32	**Log-Likelihood:**	994.52
No. Observations:	250	**AIC:**	-1985.
Df Residuals:	248	**BIC:**	-1978.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	0.0001	0.000	0.346	0.730	-0.000	0.001
x1	0.0516	0.010	5.253	0.000	0.032	0.071

Omnibus:	10.978	**Durbin-Watson:**	1.965
Prob(Omnibus):	0.004	**Jarque-Bera (JB):**	23.475
Skew:	0.063	**Prob(JB):**	7.99e-06
Kurtosis:	4.496	**Cond. No.**	35.0

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Let's compare the coefficients of *x10Y_TBY* from the three methods. We can see that WLS and bisquare method have similar coefficients for *x10Y_TBY* (WLS: 0.0516; Bisquare: 0.0515). It is easy to understand because both WLS and bisquare method give different weights to all data points whereas Huber gives different weights only after a residual is big enough.

We can also show different regression lines in a scatter plot to understand how outliers influence the regression result.

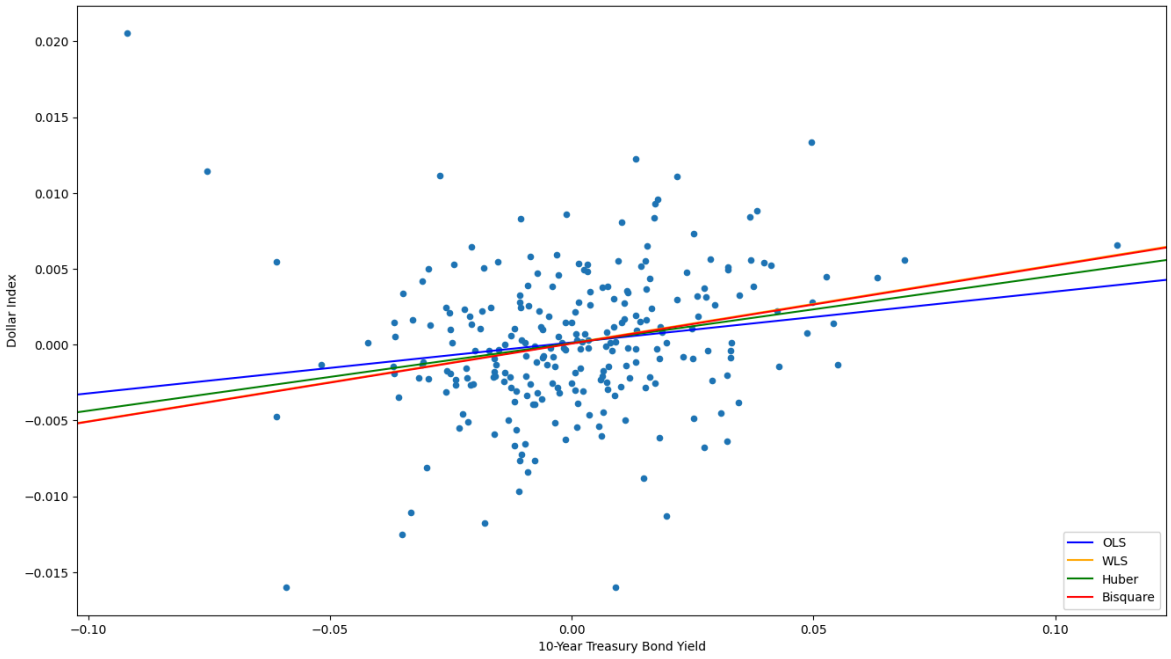
Figure 7: Scatter Plot of U.S. Dollar Index Daily Return and 10-Year Treasury Bond Yield Daily Return with Different Regression Lines

In [6]:
```
# Scatter Plot of U.S. Dollar Index Daily Return and 10-Year Treasury Bond Yield

# scatter-plot data
scatt = data_set.plot(x="X10Y_TBY", y="DXY", kind="scatter")
scatt.set(xlabel="10-Year Treasury Bond Yield", ylabel="Dollar Index")

# plot regression lines
```

```
abline_plot(model_results=model_1, ax=scatt, color="blue", label="OLS")
abline_plot(model_results=model_WLS, ax=scatt, color="orange", label="WLS")
abline_plot(model_results=rr_huber_results, ax=scatt, color="green", label="Hube
abline_plot(model_results=rr_bisquare_results, ax=scatt, color="red", label="Bis
plt.legend(loc="lower right")
plt.show()
```



From Figure 7, we can see that the WLS regression line and the bisquare regression line almost overlap. These two lines are also furthest away from outliers at the upper left of the scatter plot. The OLS regression line and Huber regression line are flatter and are pulled closer to the outliers at the upper left. The plot shows evidence again that WLS and the bisquare method apply weights to de-emphasize any data point that has a non-zero residual. OLS applies the weight of 1, and the Huber method applies weight after data points whose residuals are larger than k. Different weight schemes would lead to different regression results. After studying the above results, either WLS or the bisquare method is a better result than OLS or the Huber method.

4. Conclusion

In this lesson, we first reviewed the impact of problem outliers and leverage points on OLS regression. We then introduced robust regression to tackle outliers. There are several methods under robust regression. We talked about the most popular one: M-estimation. Specifically, we talked about the Huber method and the bisquare method under M-estimation. We finished the theory part by discussing the pros and cons of M-estimation. Lastly, we used a foreign exchange example to demonstrate how to run the Huber method and the bisquare method. We compared the results of OLS, WLS, the Huber method and the bisquare method. In the next lesson, we will discuss another branch of regression analysis: lasso regression.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 2 | LESSON 4

NON-PARAMETRIC REGRESSION

Reading Time	60 minutes
Prior Knowledge	Weighted Least Square Regression, Robust Regression
Keywords	Non-Parametric Regression, Local Averaging, Local Regression, Locally Weighted Regression, LOESS (or LOWESS), Focal Point, Span, Kernel Function, Tricube Function, Leave-One-Out Cross Validation

We discussed penalized regression in our last lesson with a large number of independent variables in a regression model. With many independent variable models, it is easy to have a multicollinearity issue, and it violates one of the ordinary least square regression (OLS) assumptions. In this lesson, we are looking into a violation of another assumption: linear dependence between the dependent variable and independent variables. When the relationship between the dependent variable and independent variables is not linear, we cannot use OLS regression to understand the relationship. We need a new method to investigate the relationship. In this lesson, we will introduce non-parametric regression. Non-parametric regression allows us to study the relationship of the dependent variable and independent variables when their relationship is not linear and usually unknown.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm

lowess_sm = sm.nonparametric.lowess
from supersmoother import SuperSmoother

plt.rcParams["figure.figsize"] = (16, 9)  # Figure size and width
# Note that supersmoother was pre-installed from !pip install --index-url https:
```

In [2]:
```
# Import data
data_set_r_working = pd.read_csv("M2. module_2_data.csv")
```

1. Non-Parametric Regression

Up until the last lesson, we assumed that the dependent variable and independent variables have a linear relationship. A linear regression model with coefficients provides a good model structure. A linear model is also easy to interpret, which makes it easier to

understand the impact of independent variables on the dependent variable. We can also use the model to conduct various statistical inference. This structure is called **parametric regression**.

However, it is not always that a dependent variable has a linear relationship with independent variables. Sometimes, we can use variable transformation to convert the dependent variable or independent variables to have a linear relationship. Sometimes, we cannot. We need a method for analysis in which we don't need to make assumptions about the linear relationship of the variables. This is when non-parametric regression comes into play. **Non-parametric regression** breaks up the data in small groups to be analyzed and then combines all results from the small data groups to form conclusions about the data overall. Non-parametric regression does not require any prior assumptions about the relationship between the dependent variable and independent variable(s).

Figure 1: Parametric Regression vs. Non-Parametric Regression

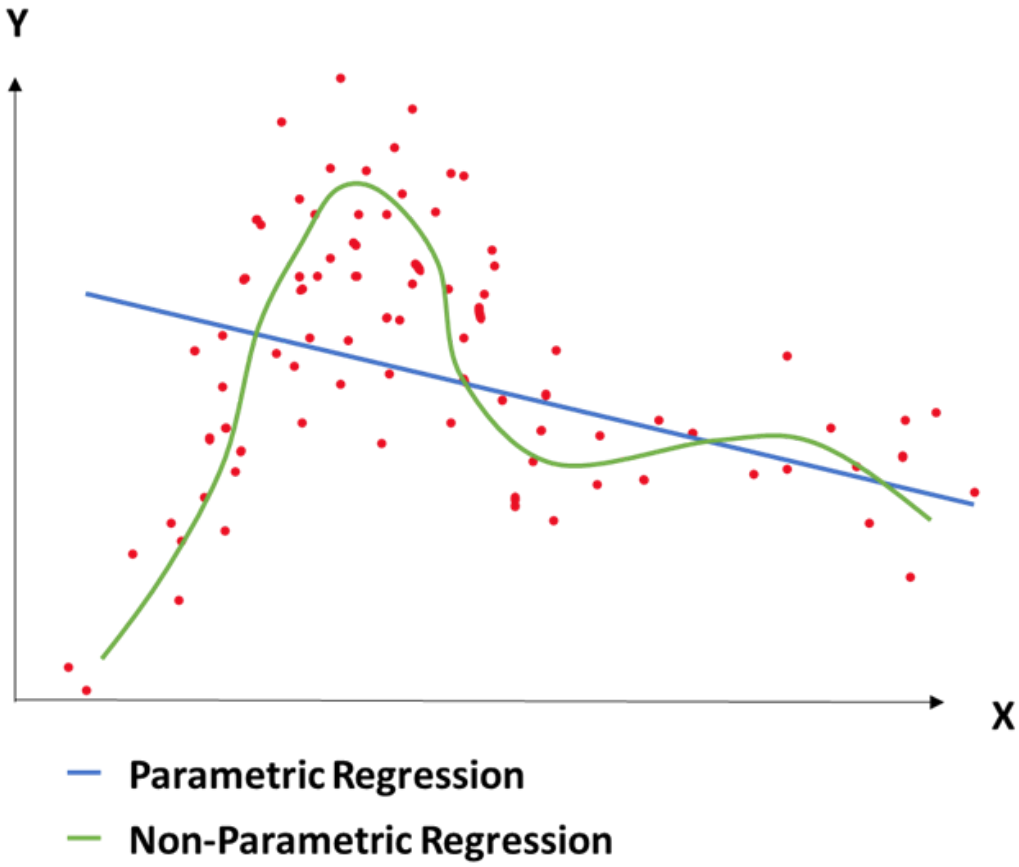


Figure 1 shows a linear regression line (parametric regression) and a non-parametric regression curve for the same dataset. We can see the non-parametric regression curve generally traces the data points better than the linear regression line in this example. We are also not sure what the relationship between X and Y is from the scatter plot. Given that the parametric regression does not seem to capture the overall curvy movement of the data, we can try to apply non-parametric regression in this case.

There are several methods of non-parametric regression. In this lesson, we will go through two classic non-parametric regression methods: local averaging and locally weighted regression (LOESS).

2. Local Averaging

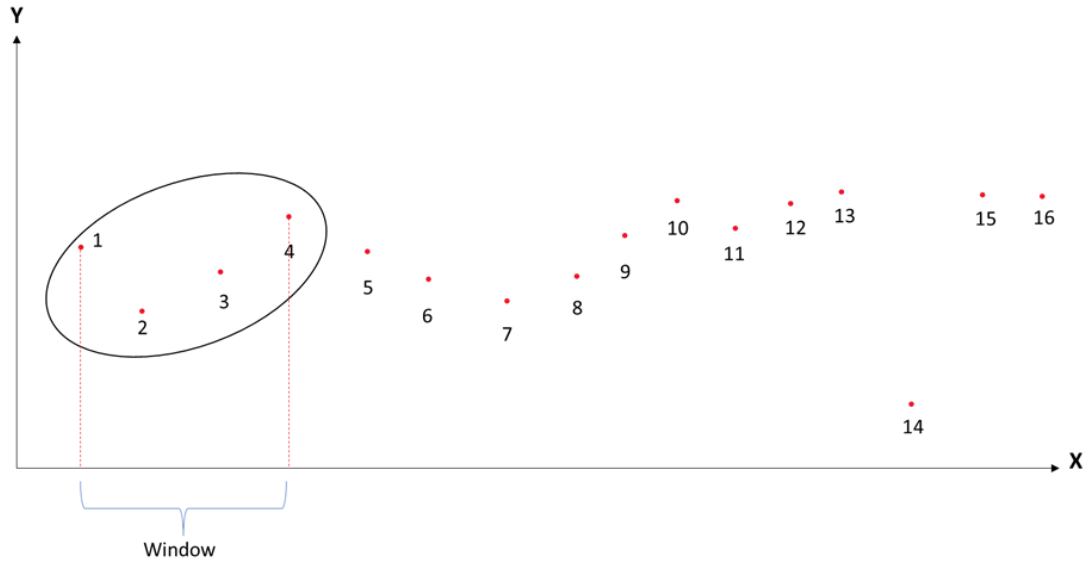
It is best to use an example to explain the local averaging method. Let's assume we have a dependent variable Y and an independent variable X. The concept of local averaging is as follows:

For a data point $A(x, y)$ in the dataset, find some nearby data points around data point A, and calculate the average of y values from data point A and nearby data points. This average y serves as the predicted y for the x from A. We repeat the same steps for all data points in the dataset. Then, we connect all predicted y values to form a local averaging curve.

Let's use Figure 2 to go through the detailed steps to derive a local averaging curve.

2.1 How Do We Select Nearby Points? M-Near Neighbor Method

Figure 2: X and Y Scatter Plot



In Figure 2, we have assigned a number to each data point so we can easily refer to a specific data point later. Let's choose data point 1 as the focal point. A **focal point** is a data point we choose as the base point to find nearby data points for analysis. How do we choose nearby data points for local averaging? We have 16 data points in this plot. We can pick a fraction of them to be included in local averaging. For now, let's choose four data points, including the focal point. Our selected points constitute 0.25 (4/16) of the total data points. This fraction is called a **span**. A **span** is a fraction that represents the number of selected data points divided by the total number of data points in the

dataset. We can use span to calculate how many data points we decide to use for local averaging. We will discuss how to choose a span level later.

Once we decide how many data points to choose, we need to have a rule for choosing them. For this example, we are using a span of 0.25, and we will choose four data points, including the focal point. How do we choose the other three nearby data points? We choose the data points that are closest to the focal point in terms of their distance on the X-axis. In the graph, we can see that data points 2, 3, and 4 are closest to focal point 1 according to their distances from focal point 1 on the X-axis. Hence, we have focal point 1 and data points 2, 3, and 4 to calculate local averaging. With these four data points, we create a frame along the X-axis to enclose these four points. We call this frame a **window**. This method of choosing data points for local analysis is called the **m-nearest neighbor method**. In our example, we select four points, so m is 4. We will also use the same data selection method in the following section.

2.2 Calculate Local Averages

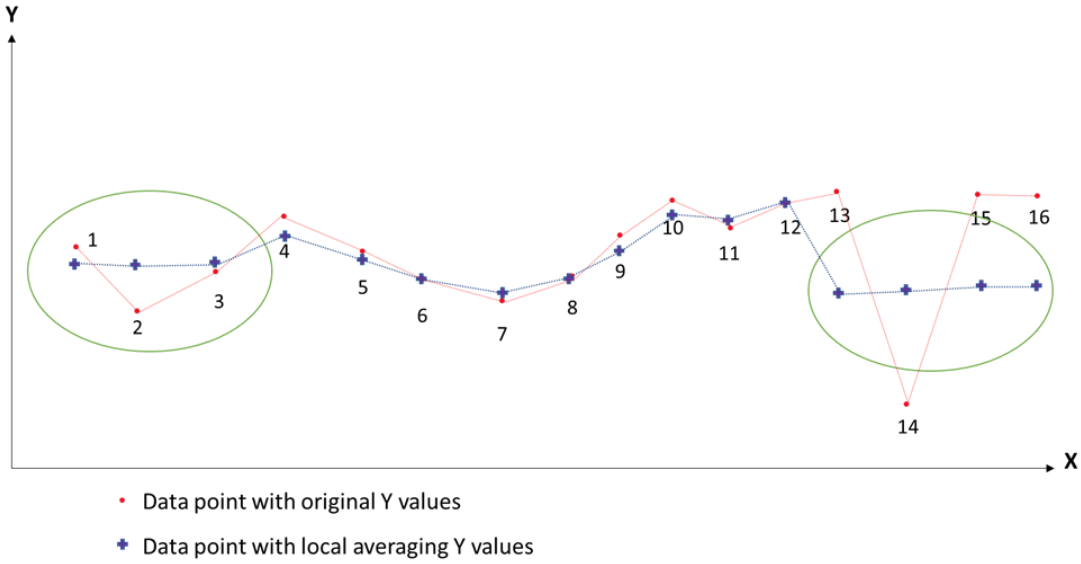
Now we know how to select nearby data points to calculate the local average for each data point. In Figure 3, we see that five focal points and their selected data points are used to calculate local averages. For the local average number for focal point 1, we calculate the average of y values ($\bar{y}$) from selected data points (data points 1, 2, 3, and 4). The predicted data point for focal point 1 is $(x, \bar{y})$. We then draw this predicted data point on the plot. In Figure 4 (below), the red dot with 1 is focal point 1, and the purple + sign near it is the predicted data point for focal point 1. We then repeat the same procedure for focal point 2 and so on until we finish all focal points in the dataset. In our case, the last focal point is data point 16.

We can see from Figure 3 that data point 1 and data point 2 actually have the same selected data points to calculate local averages. This is because for a focal point on the data boundary, its data point choices are limited to only one side of its location. Consequently, the data point choices are more likely to overlap with the next focal point. Due to this situation, the local average curve is usually flat near both ends of the data point boundaries.

Figure 3: First Five Focal Points and Their Selected Data Points for Local Averaging

Focal Point	Selected Data Points for Local Averaging
1	1, 2, 3, 4
2	1, 2, 3, 4
3	2, 3, 4, 5
4	3, 4, 5, 6
5	4, 5, 6, 7

Figure 4: Original Data Points and Predicted Y Values from Local Averaging



Now let's draw out all local average y values on the $X - Y$ scatter plot to form a local average curve. The curve is shown in Figure 4. In Figure 4, we can see the local average curve (purple line) is smoother than the curve from the original data points (red curve). We also see that both ends of the local average curve are flat because of the boundary data points issue we mentioned above.

Another issue we can see is from data point 13, the local average curve suddenly drops and stays at a lower level. This is because data point 14 is an outlier. Data point 14 is included in the local average calculation for data points 13, 14, 15, and 16, and therefore, all their average y values are lower.

The local averaging method is fairly simple. However, it has the two problems we mentioned above: The curve tends to be flat on both ends, and the curve can be distorted by the existence of outliers. The next method we are going to introduce will correct these issues.

2.3 How Do We Choose a Span Level for Local Averaging?

There are several ways to select a span level for local averaging. The first one is to draw several local average curves with different span levels and pick the one that not only captures the data's general curvy movement but also has a smooth pattern.

The more formal method is to use leave-one-out cross validation. The idea of leave-one-out cross-validation is similar to k-fold cross-validation. Instead of creating k groups, leave-one-out cross validation takes one data point out as the holdout set, calculates local averages with the given span level for the training set, and calculates the sum of squared residuals. Then, you repeat the same process with a different holdout point until all data points have been treated as a holdout point. You then calculate the average sum of squared residuals. This is the average sum of squared residuals for this given span level. You then repeat the whole process with another span level. After collecting enough data from several iterations as described above, you will get several pairs of span level

and the average sum of squared residuals. You can use this information to find the optimal span level that minimizes the average sum of squared residuals.

Most statistical applications can conduct leave-one-out cross-validation for you automatically. Please check the application you use for details.

3. Locally Weighted Regression

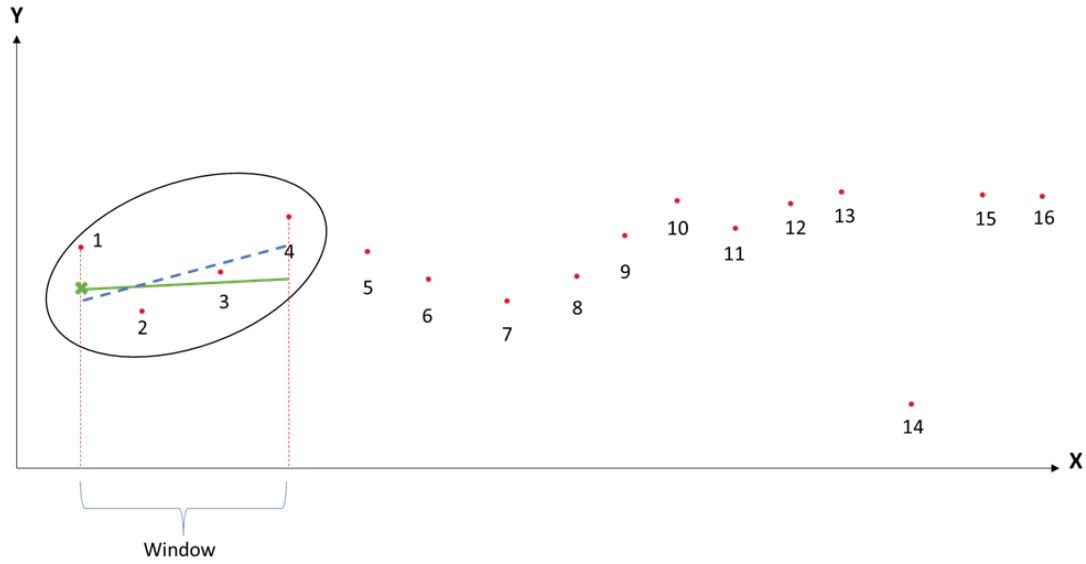
Locally weighted regression is another non-parametric regression that is aimed to control the issues we discussed in the local averaging section.

3.1 Overview of Local Regression and Local Weighted Regression

In last section, we talked about using m-near neighbor method to create small data groups around focal points to run local averaging. We will use the same m-near neighbor method to select nearby data points around a focal point for local regression. Instead of calculating the average of y values from selected data points in local averaging, we will run a regression on the selected data point to get the predicted y value for the focal point. This regression method is called **local regression**. We repeat the same process across all the datapoints until all data points have been treated as a focal point. Then, we create a local regression curve by connecting all predicted y values from each local regression model. If we weight all the selected data points in a window and run a local regression, this regression is called **locally weighted regression**. We will follow the same logic as above to create a locally weighted regression curve. This is the basic concept of locally weighted regression. Now, let's go through each step to generate a locally weighted regression curve in detail.

3.2 Basics of Locally Weighted Regressions

Figure 5. X and Y Scatterplot



Let's continue to use focal point 1 and 4 to explain locally weighted regression. Figure 5 shows these data points in the $X - Y$ scatter plot. Before running a regression for the selected data points, we need to weight each data point. How? We weigh each data point by how close they are to the focal point. The closer a point is to the focal point, the more it is weighted. The farther away a point is from the focal point, the less it is weighted. The next point with the shortest distance on the X-axis from the focal point gets the second highest weight. The point that is furthest away from the focal point in terms of distance on the X-axis will get the smallest weight. In our example, focal point 1 gets the highest weight, followed by data point 2, and then data point 3. Data point 4 gets the lowest weight. Once we find the weights for each data point, we will use the weighted least square method to run the regression.

How do we get the weight for each data point? We us a function called kernel function. A **kernel function** gives the largest weight to the observations closest to the focal point and reduces the weight as the distance between an observation and the focal point grows. There are several kernel functions to choose from, but one popular choice is the **tricube function**:

$$K(z_i) = \begin{cases} (1 - |z_i|^3)^3, & \text{if } |z_i| < 1 \\ 0, & \text{if } |z_i| \geq 1 \end{cases}$$

Where $z_i = \frac{X_i - X_{\text{focal point}}}{h}$ and h is the width of the window

Now we can use the weights from the kernel function to estimate locally weighted regression. Our weighted least square objective function is as follows:

$$\sum_{i=1}^{4} k(z_i)(Y_i - \beta_0 - \beta_1 X_i)^2$$

In Figure 5, the green solid line is the locally weighted regression line. The blue dotted line is from an OLS regression. We can see the locally weighted regression line is flatter compared to the OLS line because it gives less weight to data point 4. From the locally weighted regression, we can obtain the predicted y value for focal point 1 from the model. It is marked as the green cross in Figure 1.

We will now move on to assign data point 2 as the focal point and repeat the whole process in order to get the predicted y value for focal point 2. In Figure 3 below, we show the first five focal points in the graph and their corresponding data points for running a locally weighted regression.

Figure 6: First Five Focal Points and Their Corresponding Data Points for Locally Weighted Regression

Focal Point	Selected Data Points for Locally Weighted Regression
1	1, 2, 3, 4
2	1, 2, 3, 4

Focal Point	Selected Data Points for Locally Weighted Regression
3	2, 3, 4, 5
4	3, 4, 5, 6
5	4, 5, 6, 7

We can see for focal point 1 and focal point 2, the same data points are selected for locally weighted regression. This is the same situation in local averaging. However, data points in the focal point 1 group will receive different weights compared to the weights in focal point 2 for locally weighted regression. The same data point will have a different distance on the X-axis to the focal point in different focal point windows. This is critical for locally weighted regression: Because weight changes as the focal point changes, you don't get the same locally weighted regression from focal point 1 and focal point 2. This makes locally weighted regression different from local averaging. In local averaging, data points in the focal point 1 group and focal point 2 group receive the same weights, so the averages of y values are the same. In locally weighted regression, the same data point in different focal point groups will receive different weights. Locally weighted regression can correct the flattening curve issue near the data boundary in local averaging.

Once we conduct locally weighted regression for each point in the dataset, we can obtain the predicted y value for each data point.

Figure 7: Original Data Points and Predicted Values from Locally Weighted Regression

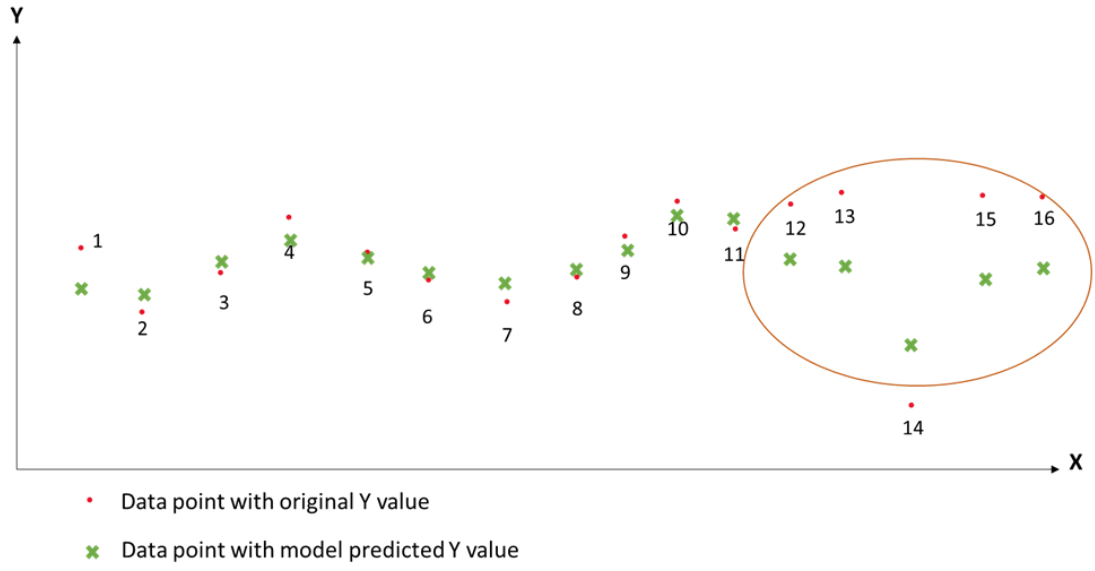


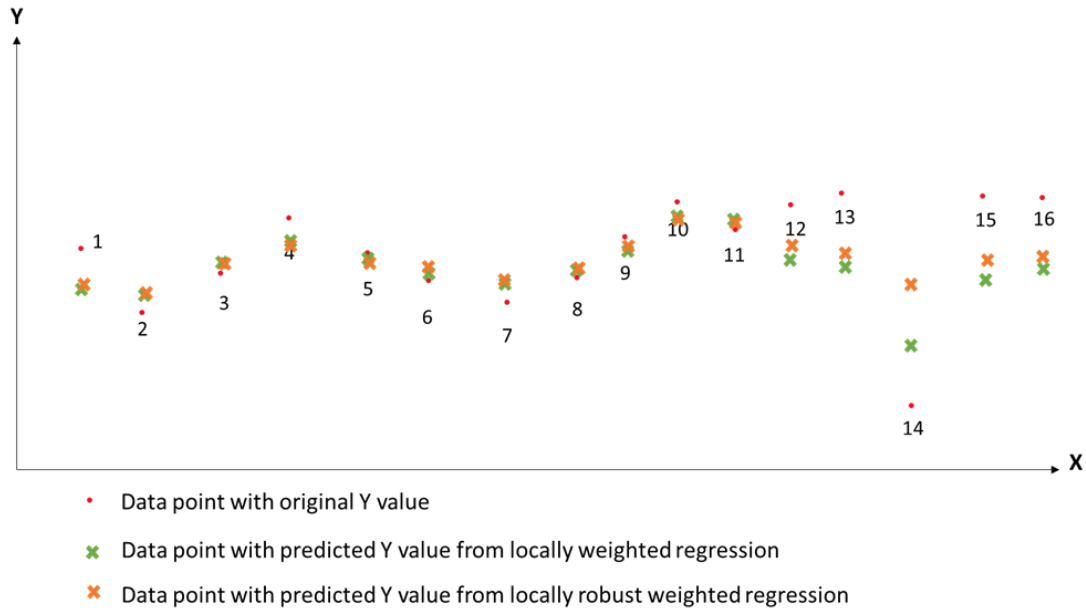
Figure 7 shows the data points with the original y values and their predicted y values from locally weighted regression. We can see the data points with predicted y values form a smoother pattern than the data points with the original y values. However, for data points 12, 13, 14, 15, and 16, the predicted y values all got pulled down from the

original y values. This is because the locally weighted regression for these data points involves data point 14, which is an outlier.

3.3 Handle Outliers for Locally Weighted Regression

So how do we handle the issue of outliers in locally weighted regression? We apply the robust regression technique we learned from Lesson 2 to handle the outlier issue. Once we get all the predicted y values for all data points in the dataset, it is common to run the m-estimation bisquare method on the predicted data points (original x and predicted y) to mitigate outlier impact on the predicted curve. We usually run robust regression a couple of times to make sure the smooth curve is stable.

Figure 8: Locally Weighted Regression without Robust Regression and with Robust Regression



From Figure 8, we can see the curve formed by the data points from locally robust weighted regression (the orange crosses) is even smoother than the curve from locally weighted regression (the green crosses).

3.4 Locally Weighted Polynomial Regression

In the last section, we assumed our locally weighted regression objective function to be a simple linear regression as follows:

$$\sum_{i=1}^{4} k(z_i)(Y_i - \beta_0 - \beta_1 X_i)^2$$

There is another option for the objective function. Instead of a linear formula, we can also use a polynomial formula as follows:

$$\sum_{i=1}^{4} k(z_i)(Y_i - \beta_0 - \beta_1 X_i - \beta_2 X_i^2 - \ldots - \beta_p X_i^p)^2$$

If we choose a degree of two for the polynomial formula, the objective function will be as follows:

$$\sum_{i=1}^{4} k(z_i)(Y_i - \beta_0 - \beta_1 X_i - \beta_2 X_i^2)^2$$

When we use a locally weighted polynomial regression with a degree of two to run the model, we use a method called **LOESS: locally weighted scatterplot smoothing**. LOESS has an old name called LOWESS, as LOWESS is an older version of LOESS. In some statistical applications, you might find both LOESS and LOWESS available. Although they are the same method, some applications will have different default settings for LOESS and LOWESS that would make model results on the same dataset different. For example, R has both LOESS and LOWESS. LOWESS can only run locally weighted **linear** regression while LOESS can run both locally weighted linear regression and locally weighted polynomial regression. In R, the default setting for LOESS is locally weighted polynomial regression with a degree of two. Please check your statistical application for model specification.

3.5 How Do We Choose Span?

Span in locally weighted regression has the same meaning as in local averaging. Span is the ratio of the number of data points used to run a locally weighted regression to the total number of data points in the dataset. Span has a value between 0 and 1. If the span is too small, the local regression will use fewer data points. The fit will be more accurate for the data points in the regression. However, the local regression results among different windows will be very different, creating more variance among different local regression lines. The curve will be wigglier if the span is small. When the span is too big, you include more data for a local regression. The fit will be less accurate for data points in the regression since there are more data points in the local regression now. However, the local regressions among different windows will be more similar, resulting in less variance. The curve will also be a lot smoother. Therefore, it is important to find a span that balances the variance and bias.

As for finding a good span number, the common practice is to use visual inspection. You can draw LOESS with various span levels and pick the one that balance the smoothness of the curve and the prediction of the data.

The formal method to choose the optimal span is also similar to local average. Leave-one-out cross-validation is widely available in most statistical applications. Another similar cross-validation method called generalized cross validation is also a good choice and is also available in many applications. Leave-one-out cross-validation and generalized cross-validation (GCV) are more rigorous in finding an optimal span. Choose the method that is available in the application you decide to use.

3.6 Pros and Cons of LOESS

LOESS is a popular non-parametric method. First of all, you don't need to have any assumptions about the relationship between a dependent variable and an independent variable. It is also very easy to implement in any statistical software.

However, unlike least square regressions, LOESS doesn't produce coefficients. It is hard to interpret the model results and use the model output to run statistical inference. Another issue is if the dataset is not large enough, LOESS will not produce a satisfactory result. LOESS needs enough data points to run locally weighted regressions to make sense of the whole model.

Not having enough data also makes the tail of the LOESS curve less accurate. This is because in the locally weighted regression method, the regressions near both ends of the dataset usually have fewer data points than the regressions in the middle of the dataset. Hence, if the dataset is not large enough, the tail of the dataset will have less accurate regressions.

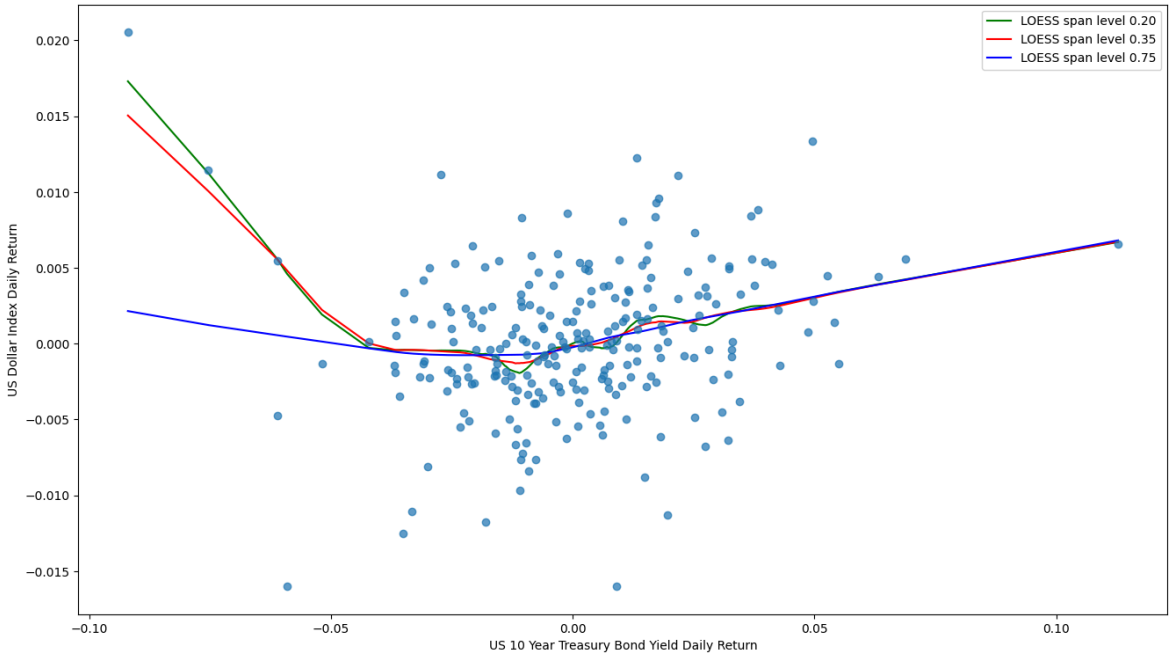
4. Foreign Exchange Market and Non-Parametric Regression

Now, let's use local averaging and LOESS on our U.S. Dollar Index example. We will still use U.S. 10-Year Treasury Bond Yield daily return as the independent variable and U.S. Dollar Index daily return as the dependent variable.

Figure 9: U.S. Dollar Index Daily Return vs. U.S. 10-Year Treasury Bond Daily Return Scatter Plot and LOESS with Various Spans

In [3]:
```
# Compute a lowess smoothing of the data at 0.20, 0.35 and 0.75 span values
smoothed20 = lowess_sm(
    endog=data_set_r_working.DXY, exog=data_set_r_working.X10Y_TBY, frac=0.20, i
)
smoothed35 = lowess_sm(
    endog=data_set_r_working.DXY, exog=data_set_r_working.X10Y_TBY, frac=0.35, i
)
smoothed75 = lowess_sm(
    endog=data_set_r_working.DXY, exog=data_set_r_working.X10Y_TBY, frac=0.75, i
)

# Plot the fit lines
plt.errorbar(data_set_r_working.X10Y_TBY, data_set_r_working.DXY, fmt="o", alpha
plt.plot(smoothed20[:, 0], smoothed20[:, 1], c="g", label="LOESS span level 0.20
plt.plot(smoothed35[:, 0], smoothed35[:, 1], c="r", label="LOESS span level 0.35
plt.plot(smoothed75[:, 0], smoothed75[:, 1], c="b", label="LOESS span level 0.75
plt.xlabel("US 10 Year Treasury Bond Yield Daily Return")
plt.ylabel("US Dollar Index Daily Return")
plt.legend()
plt.show()
```



In figure 9, there are 3 different LOESS curves with the span levels 0.20, 0.35, and 0.75. The red curve has a span level of 0.35, which is the optimal span derived from generalized cross validation. We can see that the red curve is better than the blue line as the red curve captures more data movement. The red curve is also better than the green curve as the red curve is not too wiggly compared to the green curve.

Figure 10: U.S. Dollar Index Daily Return vs. U.S. 10-Year Treasury Bond Daily Return Scatter Plot and LOESS and Local Averaging Curves

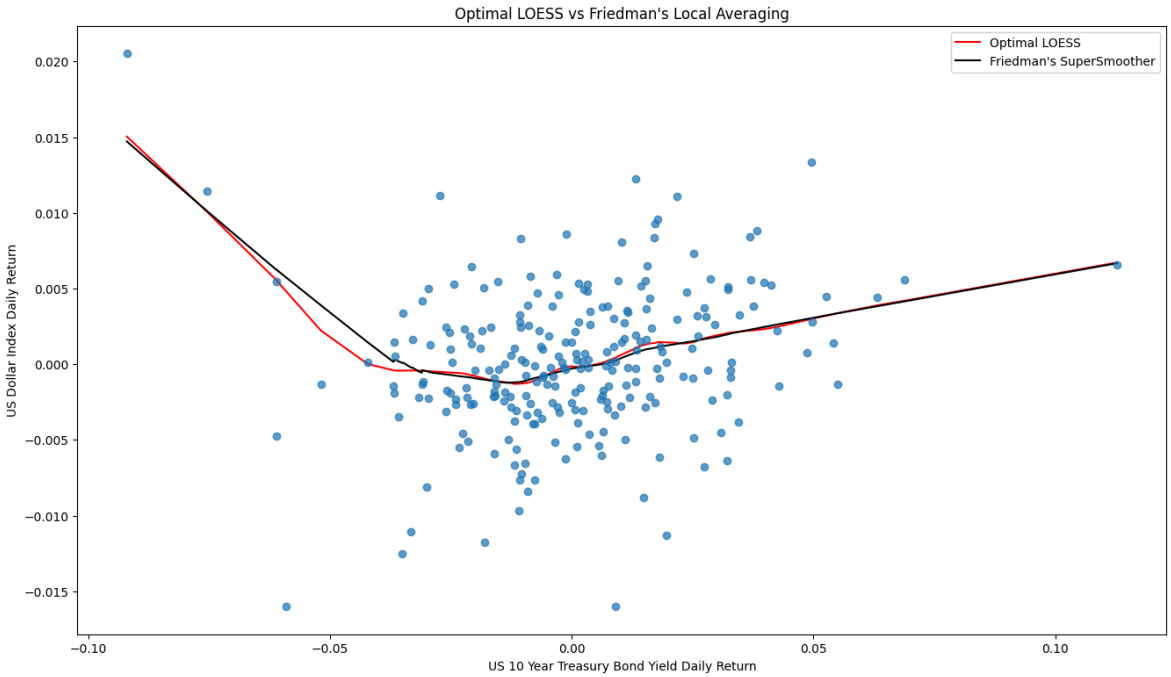
In [4]:
```
# DXY vs X10Y_TBY scatter plot and LOESS and Local Averaging Curves

# Find the smoothed fit using the Friedman's SuperSmoother model
model = SuperSmoother(alpha=None, period=None, final_span=0.138)
model.fit(
    data_set_r_working.X10Y_TBY,
    data_set_r_working.DXY,
    np.ones(len(data_set_r_working.DXY)),
)  # t, y, dy
fit_DXY = model.predict(data_set_r_working.X10Y_TBY)

# The smoothed model of the data
temp_data = pd.DataFrame([data_set_r_working.X10Y_TBY, fit_DXY]).T
temp_data.columns = ["fit_DXY", "X10Y_TBY"]
temp_data = temp_data.sort_values(by="fit_DXY")

# Plot scatter plot, optimal LOESS and Friedman's smoothed model
plt.errorbar(data_set_r_working.X10Y_TBY, data_set_r_working.DXY, fmt="o", alpha
plt.plot(smoothed35[:, 0], smoothed35[:, 1], c="r", label="Optimal LOESS")
plt.plot(temp_data.fit_DXY, temp_data.X10Y_TBY, "-k", label="Friedman's SuperSmo

plt.title("Optimal LOESS vs Friedman's Local Averaging")
plt.xlabel("US 10 Year Treasury Bond Yield Daily Return")
plt.ylabel("US Dollar Index Daily Return")
plt.legend()
plt.show()
```



In figure 10, we compare LOESS and local averaging. We chose LOESS (red curve) with the optimal span from the last example. The local averaging curve (black curve) here has the optimal span derived from leave-one-out cross-validation. We can see these two curves trace each other pretty well except on the left end of the plot. The local averaging curve is slightly flatter than LOESS on the left end. This result confirms what we learned from the last section that a local averaging curve tends to get flat on both sides of the data boundaries.

5. Shiny Application Weighted Least Square Regression and Robust Regression

In this module, we provide a Shiny application to build a weighted least square regression model and a robust regression model. The application also gives a data summary to let you know what variables are available in the application.

Click here to access the application. Select a dependent variable, independent variable(s), and year(s) to run and then press the run button. The application will generate model results for a weighted least square regression model and a robust regression model at the same time.

6. Conclusion

In this lesson, we learned that when the dependent variable and independent variables don't have a linear relationship, we use non-parametric regression to explore their relationship. We introduced local averaging and locally weighted regression. We went through the steps to derive both methods on an $X - Y$ scatter plot. We briefly talked about how to select an optimal span level. We then introduced LOESS, the most popular non-parametric regression method. In the end, we discussed the pros and cons of LOESS. The lesson ended with examples that generated a local average curve and several LOESS

curves from our foreign exchange market data. This lesson concludes our exploration on regression analysis. We studied classic OLS regression and its assumptions. We then started to challenge those assumptions and introduced alternative regression methods to handle assumption violation. We used weighted least square regression when the variances of the error terms are not constant. We then introduced robust regression when there are outliers in the dataset. We applied penalized regression when we have high-dimensional data. Lastly, we learned Non-Parametric Regression when our dependent variable and independent variables don't have a linear relationship. In the next module, we will look at some variable distributions and their applications.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 2 | LESSON 1

OLS REGRESSION ASSUMPTIONS AND WEIGHTED LEAST SQUARE REGRESSION

Reading Time	60 minutes
Prior Knowledge	OLS regression analysis, Linear Algebra
Keywords	Homoscedasticity, Heteroskedasticity, Independent and identically distributed, Weighted least square regression, Breusch-Pagan Test

In the last module, we discussed the applications of linear regression models and some of their assumptions. We learned how to handle multicollinearity when independent variables are highly correlated. We also went through methods to transform variables when they don't meet the assumptions in a regression model. In this module, we will dive deeper into and offer variations of regression analysis. We will first review main assumptions for an ordinary least square model. Then, we will focus on the heteroskedasticity of a regression model and introduce the weighted least square model as a way to handle the issue.

We are also introducing a new working example in this module. We will analyze some factors that might impact the foreign exchange markets. We will use different regression methods we learn in each lesson to conduct the analysis.

In [1]:
```
# Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms

plt.rcParams["figure.figsize"] = (14, 8)  # Figure size and width
```

In [2]:
```
# Import data
data_set = pd.read_csv("M2. module_2_data.csv")
```

1. Assumptions of Ordinary Least Square Regression

Ordinary least square (OLS) regression models are based on the following assumptions.

1.1 Linear Relationship

The OLS regression model assumes that the dependent variable has a linear relationship with independent variables. If Y is the dependent variable and X_1, X_2 and X_3 are independent variables and e is the error term, we can write an OLS regression model as follows:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + e$$

From Module 1, we learned how to use variable transformation methods to ensure this relationship exists.

1.2 No Multicollinearity

This assumption states that independent variables should be linearly independent to each other. In reality, many independent variables are correlated to each other. Some might have low correlations and some might have high correlations. We learned from the last module that if independent variables are highly correlated, we can check their correlations or VIFs and drop variables. We can also apply principal component analysis or factor analysis to address the issue.

1.3 Independent Error Terms

This assumption for OLS means that the error terms from each observation will be independent of each other.

Assume we have these two observations from the dataset $(y_1, X_{11}, X_{21}, X_{31})$ and $(y_2, X_{12}, X_{22}, X_{32})$. We can plug these two observations into an OLS regression model system.

$$y_1 = \beta_0 + \beta_1 X_{11} + \beta_2 X_{21} + \beta_3 X_{31} + e_1$$
$$y_2 = \beta_0 + \beta_1 X_{12} + \beta_2 X_{22} + \beta_3 X_{32} + e_2$$

This assumption states that e_1 and e_2 are independent of each other. If they are not independent, we call them **autocorrelated**.

1.4 Error Terms are Normally Distributed

From our example, this means that e_1 and e_2 have to be normally distributed.

1.5 Expected Values of Error Terms are 0

From our example, the expected values of e_1 and e_2 have to be 0.

1.6 Error Terms Have the Same Variance (Homoscedasticity)

When the variances of error terms are constant, we call the error terms **homoscedastic.** If the error terms are not constant, we call them **heteroskedastic.**

With all the above assumptions about error terms, we usually call the error terms **IID: independent and identically distributed**. With the assumptions of 1.4, 1.5, and 1.6, we can summarize the error terms as follows:

$$e \sim N(0, \sigma^2)$$

Now we can write the multiple regression mean function as follows:

$$E(Y|X) = \beta X$$

And variances of the errors are constant.

$$\mathrm{Var}(Y|X) = \mathrm{Var}(e|X) = \delta^2$$

Because the variance function is for multiple regression, $\mathrm{Var}(e)$ is actually a covariance matrix.

$$\mathrm{Var}(e) = \begin{pmatrix} \delta^2 & 0 & 0 & \cdots & 0 \\ 0 & \delta^2 & 0 & \cdots & 0 \\ 0 & 0 & \delta^2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \delta^2 \end{pmatrix}$$

We can see the values on the diagonal of the matrix are the constant variance, and the values off the diagonal of the matrix are 0. Because the assumption error terms are independent, the covariances of different error terms are 0.

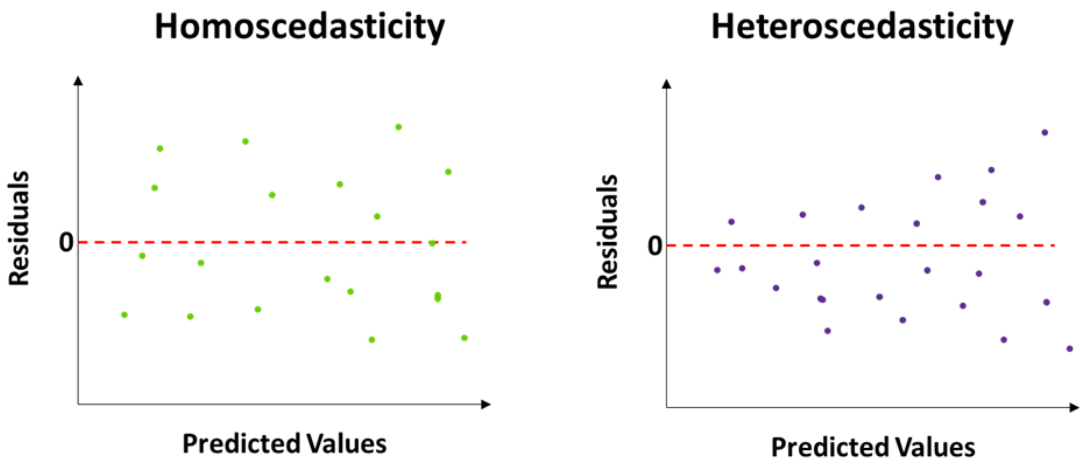
Now we want to see what happens when any of the above assumptions do not exist. In the last module, we talked about how to handle the situations when the linear relationship and low multicollinearity assumptions are violated. In the following section, we will turn our focus on the assumption of homoscedasticity. We want to see how to detect when the homoscedasticity assumption does not hold up and what method we can use to run a regression analysis when heteroskedasticity exists. Particularly, we will introduce a weighted least square regression to handle heteroskedasticity.

2. Weighted Least Square Regression

2.1 How to Detect Heteroskedasticity

As we mentioned in the last section, heteroskedasticity occurs when error terms in a regression analysis are not constant among observations. After running an OLS regression model, we can draw a scatter plot between predicted values vs. residuals to check for homoscedasticity (or heteroskedasticity). **Residuals** are the differences between measured values of the dependent variables and predicted values of dependent variables from the OLS regression model.

Figure 1: Homoscedasticity vs. Heteroskedasticity



From Figure 1, we can see the residuals scatter around the 0 line evenly in the homoscedasticity graph on the left. It means the variance for residuals is very constant along the 0 line. However, in the heteroskedasticity graph on the right, the residuals are more concentrated when the predicted values are small, and the residuals are more spread out when the predicted values are large. The data shows a cone or a megaphone pattern in the graph. It implies that as the predicted values increase, the variances increase too. The variances are not constant.

Graphs are visual aids. Sometimes, the graphs can make it difficult to spot the issue of heteroskedasticity. There are several formal methods to detect heteroskedasticity. The **Breusch-Pagan (BP) test** is a formal statistical test to detect the issue.The idea of the BP test is that it assumes that the variance and independent variables have a linear relationship. The test then runs a regression of the squared residuals from the original regression on the independent variables. Then, a BP test checks the following hypothesis:

$$H_0 : \text{Homoscedasticity exists}$$

$$H_1 : \text{Heteroskedasticity exists}$$

This is a *Chi-square* test. If the p-value is less than the significant level of 0.05, we reject the null hypothesis. The residuals are not constant, so there is a heteroskedasticity issue.

There are other tests to check for heteroskedasticity. One of them is the **White test,** which tests the same hypothesis as the BP test. However, the White test does not assume a linear relationship between the variance and independent variables.

2.2 Basics of Weighted Least Square Regression

When the error term variances are not constant, the covariance matrix for n observations will be like the following:

$$\mathrm{Var}(e) = \begin{pmatrix} \delta_1^2 & 0 & 0 & \cdots & 0 \\ 0 & \delta_2^2 & 0 & \cdots & 0 \\ 0 & 0 & \delta_3^2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \delta_n^2 \end{pmatrix}$$

where $\delta_1^2 \neq \delta_2^2 \neq \delta_3^2 \neq \cdots \neq \delta_n^2$

Let's assume the variance of the error term of observation i is as follows:

$$\mathrm{Var}(e_i) = \delta_i^2 = \frac{\sigma^2}{w_i}$$

Where $w_1, \cdots, w_n$ are known positive numbers

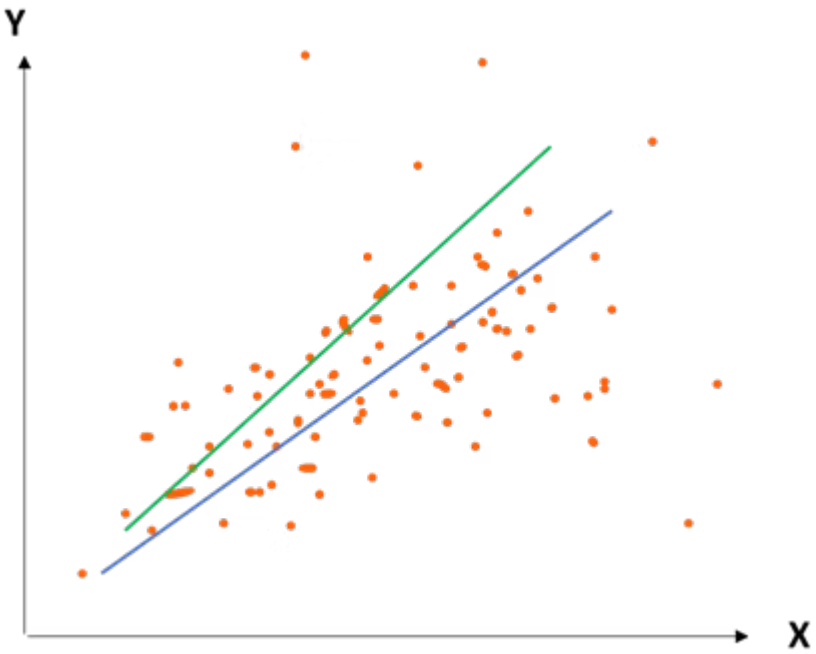
For weighted least square regression, we need to solve for coefficients by minimizing the following sum of weighted squared residuals.

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^{n} w_i (Y_i - \beta_0 - \beta_1 X_i)^2$$

The above function is called an **objective function**. An **objective function** is a function we would like to maximize or minimize. Please see section 4 in this lesson for more information on how to derive the coefficients of weighted least square regression.

We can see the weight w_i is inversely proportional to the variance of each observation. It means for an observation that is highly variable, it will be given a smaller weight for its weighted squared residual. Hence, this observation will have less importance in the minimization process. For an observation that has small variability, its weight will be larger. Its weighted squared residual will be larger, and it will be more important in the minimization process.

Figure 2: Impact of Data Weighting on Regression Lines



From Figure 2, Y is the dependent variable and X is the independent variable of a regression. Figure 2 shows the scatterplot of the two variables. From the graph, we can see the data is not homoscedastic because there is more variation of Y when X increases. If we do not apply weights to the data and run an OLS regression model, then the regression line will try to treat all data points equally. The result line will be like the green line. The green line is higher and steeper towards the data point on the top of the graph because OLS treats every point equally. However, if we apply weights to all the data points, we will reduce the impact of those points that are further away from where most of the points cluster together. The weighted least square (WLS) regression line will be like the blue line. WLS focuses more on the data points that are close to each other and gives lower weight to points that are farther away. Hence, WLS is also a good method to handle **outliers**. When there is an outlier in the dataset, by the method of WLS, it will give less weight so this outlier will have less impact on the result of the WLS regression model.

One thing to notice is that if the weight is 1, then we are basically running an OLS regression model. In the field of regression analysis, OLS is actually a special case of WLS because OLS has weights equal to 1.

2.3 How Do We Estimate Weights?

We don't always know the numbers of the weight. What should we do when the weights are unknown? In the last section, we learned that the weight is proportionally inverse to the variance of each error term.

$$w_i = \frac{\sigma^2}{\sigma_i^2}$$

Since w_i is the weight and σ^2 is an unknown constant and we only care about relationship among weights, not the scale, we can assume σ^2 to be 1. As such, we can try

to estimate the following weight function.

$$w_i = \frac{1}{\sigma_i^2}$$

However, we still don't know the variance for each error term of the observation (σ_i^2). We will need to estimate the variances of error terms.

Here are the steps to run a WLS regression model:

Step 1: Run an OLS regression model.

Step 2: Draw a scatter plot with fitted values from OLS on the X axis and residuals from OLS on the Y axis.

Step 3: Inspect the scatter plot to see if there is a cone or megaphone pattern in the plot to indicate heteroskedasticity. You can also use the **Breusch-Pagan test** to check for heteroskedasticity, as discussed in the last section.

Step 4: If you determine that there is heteroskedasticity in the OLS model, you will have to estimate the weights. We know from the last section that the weight for each observation is the inverse of the variance. We can use squared residuals from OLS to estimate the variances. It also means to use residuals from OLS to estimate the standard deviations. In order to estimate squared residuals, we will run an OLS regression model with absolute values of the residuals as the dependent variable and the fitted values from OLS model in **Step 1** as an independent variable as follows:

$$|\text{residuals from OLS}| = 1 + \gamma \cdot \text{fitted values from OLS} + e$$

We will use the fitted value from the above OLS regression as the proxy for standard deviation. Variance is the squared standard deviation. We then plug the estimated standard deviation into the following equation to get the weight variable.

$$\text{weight} = \frac{1}{\text{standard deviation}^2}$$

Step 5: Once we get the weight variable, we can plug this weight variable into any statistical software to run a weighted least square regression model.

3. Foreign Exchange Market Analysis Using Weighted Least Square Regression

Now, let's look at an example to learn how to run weighted least square regression.

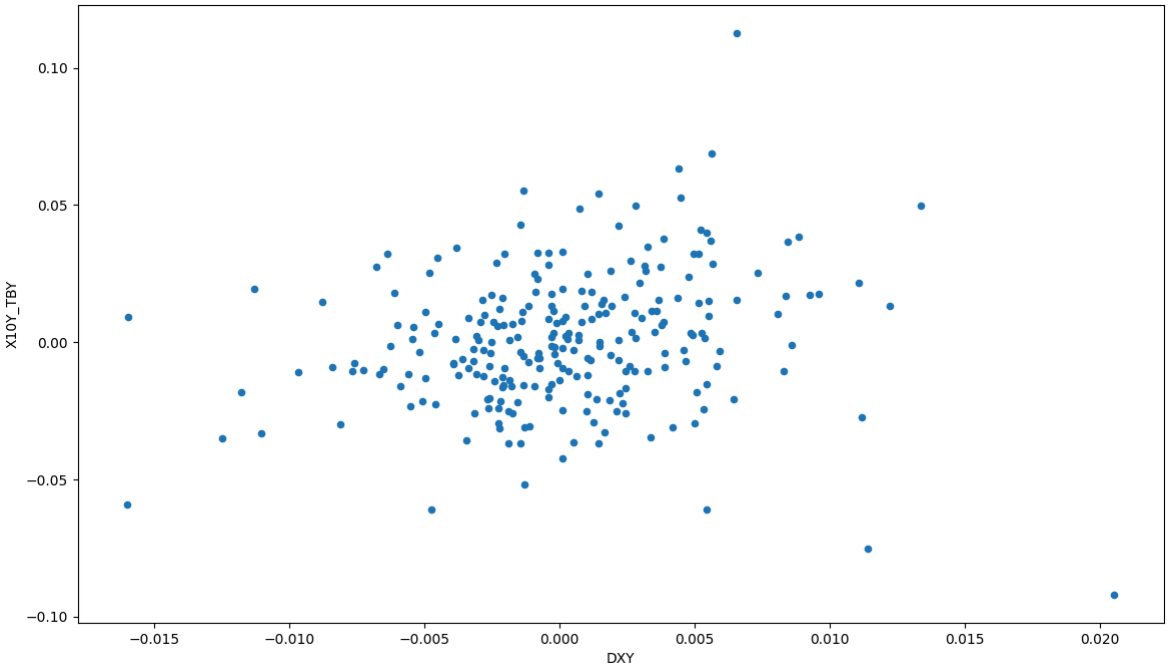
We will use WLS to understand how the U.S. 10-year treasury bond yield would influence the U.S. Dollar Index. Both variables are converted to daily returns for analysis. We use the daily data from 2016. Here are their variable names.

- *DXY*: US Dollar Index daily return
- *X10Y_TBY*: 10-Year Treasury Bond Yield daily return

First, let's draw a scatter plot to see if the two variables have any relationship.

Figure 3: U.S. Dollar Index Daily Return and 10-Year Treasury Bond Yield Daily Return Scatter Plot

In [3]:
```
# Create scatter plot
scatt = data_set.plot.scatter(x="DXY", y="X10Y_TBY")
scatt.set(xlabel="DXY", ylabel="X10Y_TBY")
plt.show()
```



From Figure 3, we can see two interesting patterns. First, they seem to have a positive relationship. We can see as *DXY* increases, *X10Y_TBY* increases too. Secondly, as *DXY* increases, the data points spread out more. It forms a cone or megaphone shape. It indicates that there might be a heteroskedasticity issue.

Now, let's run an OLS regression with *DXY* as the dependent variable and *X10Y_TBY* as the independent variable.

Figure 4: OLS Regression Result

In [4]:
```
# OLS model
model_1 = smf.ols("DXY ~ X10Y_TBY", data=data_set).fit()
model_1.summary()
```

Out[4]:

OLS Regression Results

Dep. Variable:	DXY	**R-squared:**	0.030
Model:	OLS	**Adj. R-squared:**	0.026
Method:	Least Squares	**F-statistic:**	7.738
Date:	Fri, 05 Dec 2025	**Prob (F-statistic):**	0.00582
Time:	18:18:12	**Log-Likelihood:**	986.78
No. Observations:	250	**AIC:**	-1970.
Df Residuals:	248	**BIC:**	-1963.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0001	0.000	0.474	0.636	-0.000	0.001
X10Y_TBY	0.0336	0.012	2.782	0.006	0.010	0.057

Omnibus:	26.741	**Durbin-Watson:**	1.909
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	94.918
Skew:	0.322	**Prob(JB):**	2.45e-21
Kurtosis:	5.949	**Cond. No.**	40.7

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [5]:
```
# Parameters with high precision
print(model_1.summary2().tables[1])
```

```
              Coef.  Std.Err.         t     P>|t|    [0.025    0.975]
Intercept  0.000141  0.000297  0.474101  0.635845 -0.000444  0.000725
X10Y_TBY   0.033568  0.012067  2.781725  0.005823  0.009800  0.057336
```

From Figure 4, we can see that *DXY* and *X10Y_TBY* have a positive coefficient, which is consistent with what we saw in the scatter plot. The coefficient is also significant. The R^2 is not very good for this simple regression model. Now, let's check the scatter plot for residuals and fitted values of this OLS model to see if there is a heteroskedasticity issue.

Figure 5: Scatter Plot for OLS Fitted Values and OLS Residuals

In [6]:
```
# Scatter Plot for OLS Fitted Values and OLS Residuals

# Fitted values and residuals
model_fitted_y = model_1.fittedvalues
model_resid_y = model_1.resid

# Plot
```

```
plt.scatter(x=model_fitted_y, y=model_resid_y)
plt.title("OLS Residuals vs OLS Fitted Values")
plt.xlabel("OLS Fitted values")
plt.ylabel("OLS Residuals")
plt.show()
```

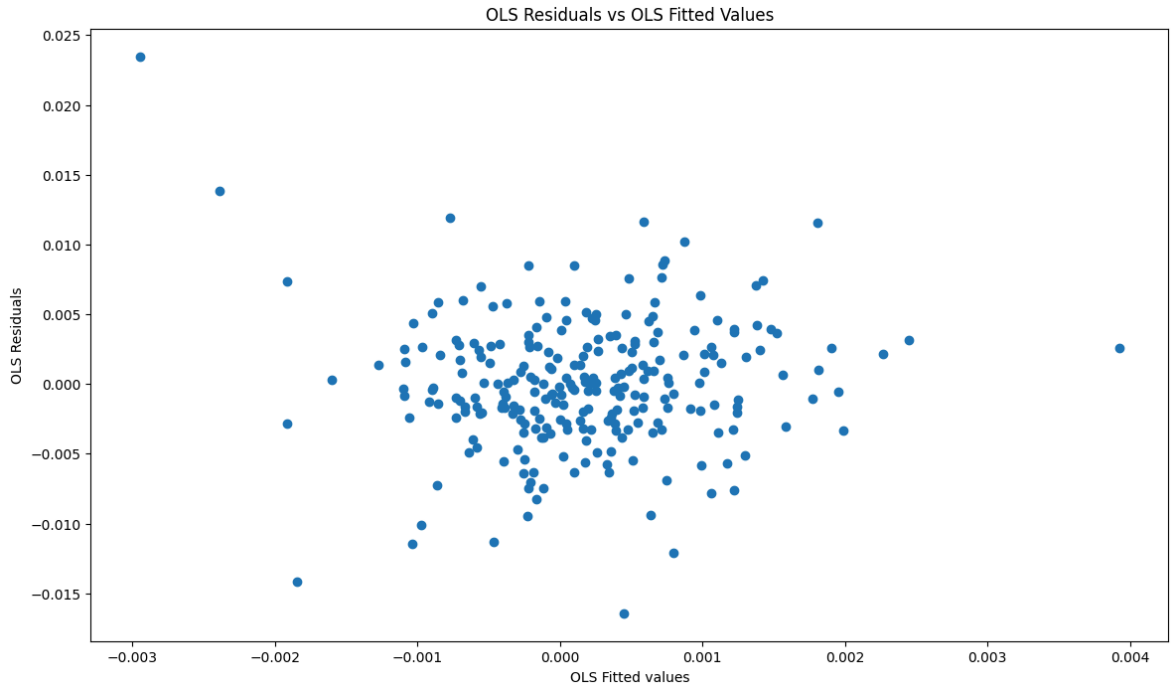


Figure 5 shows that there is more variation in residuals when fitted values are negative than when fitted values are positive, especially when fitted values are below -0.002. The scatter plot indicates there might be a heteroskedasticity issue. Let's run a Breusch-Pagan test to double-check the hypothesis.

Figure 6: Breusch-Pagan Test Result

In [7]:
```
# Breusch-Pagan Test
name = ["Lagrange multiplier statistic", "p-value", "f-value", "f p-value"]
test = sms.het_breuschpagan(model_1.resid, model_1.model.exog)
pd.DataFrame(test, index=name, columns=[""])
```

Out[7]:

Lagrange multiplier statistic	13.897238
p-value	0.000193
f-value	14.597521
f p-value	0.000168

The test result from Figure 6 also confirms the existence of heteroskedasticity because the p-value is less than 0.05. Based on the above information, we will run the model with weighted least square, with the weight generated from the formula from the last section.

Figure 7: WLS Result

In [8]:
```
# WLS regression result

# Add Absolute residuals and fitted values to dataset columns
```

```
data_set["abs_residuals"] = np.abs(model_1.resid)
data_set["fitted_values"] = model_1.fittedvalues

# Fit OLS model with absolute residuals and fitted values
model_temp = smf.ols("abs_residuals ~ fitted_values", data=data_set).fit()

# Compute weights and add it to the data_set column
weights = model_temp.fittedvalues
weights = weights**-2
data_set["weights"] = weights

# Fit WLS model
Y = data_set["DXY"].tolist()
X = data_set["X10Y_TBY"].tolist()
X = sm.add_constant(X)  # add a intercept point

model_WLS = sm.WLS(Y, X, data_set["weights"]).fit()
model_WLS.summary()
```

Out[8]:

WLS Regression Results

Dep. Variable:	y	**R-squared:**	0.100
Model:	WLS	**Adj. R-squared:**	0.096
Method:	Least Squares	**F-statistic:**	27.59
Date:	Fri, 05 Dec 2025	**Prob (F-statistic):**	3.23e-07
Time:	18:18:12	**Log-Likelihood:**	994.52
No. Observations:	250	**AIC:**	-1985.
Df Residuals:	248	**BIC:**	-1978.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	0.0001	0.000	0.346	0.730	-0.000	0.001
x1	0.0516	0.010	5.253	0.000	0.032	0.071

Omnibus:	10.978	**Durbin-Watson:**	1.965
Prob(Omnibus):	0.004	**Jarque-Bera (JB):**	23.475
Skew:	0.063	**Prob(JB):**	7.99e-06
Kurtosis:	4.496	**Cond. No.**	35.0

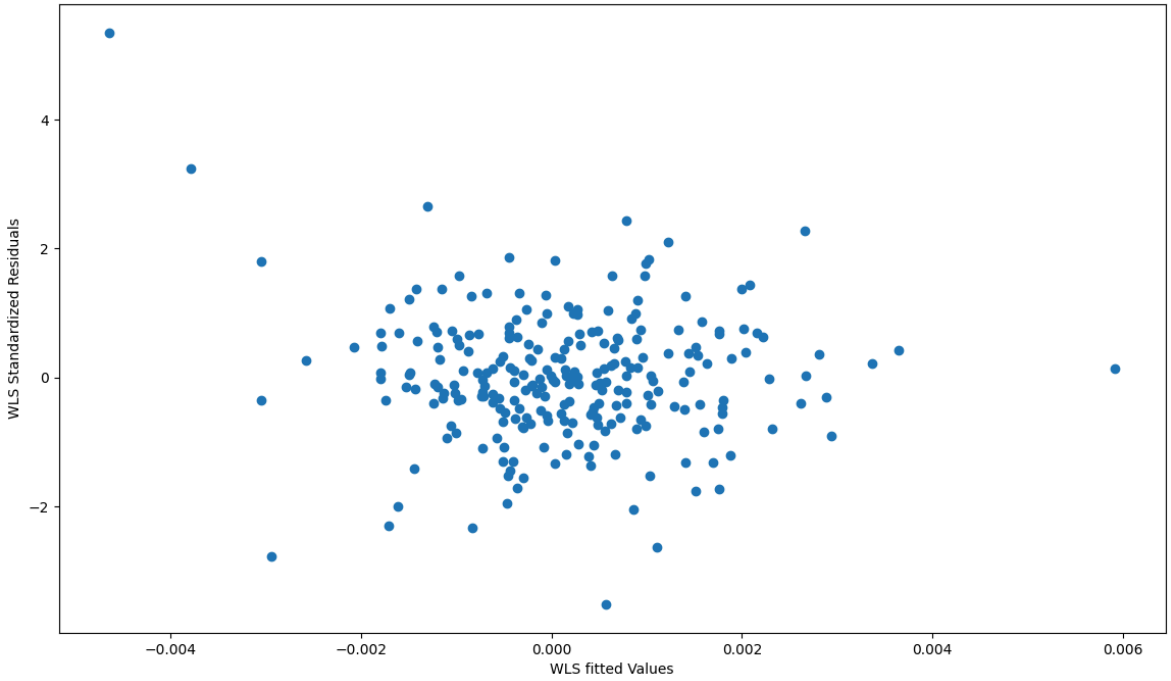
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

From Figure 7, we can see *X10Y_TBY* still has a positive coefficient and the coefficient is still significant. The R^2 also improves a lot from 0.03 to 0.10. Now, let's look at the residuals and fitted values from WLS to see if we can correct the heteroskedasticity issue.

Figure 8: WLS Standardized Residuals and Fitted Values Scatter Plot

In [9]:
```
# WLS fitted values and standardized residuals
fitted_valuesWLS = model_WLS.fittedvalues
model_residWLS = model_WLS.resid
std_model_residWLS = model_residWLS / model_residWLS.std(ddof=1)

# Plot WLS fitted v. vs standardized res. scatterplot
plt.scatter(fitted_valuesWLS, std_model_residWLS)
plt.xlabel("WLS fitted Values")
plt.ylabel("WLS Standardized Residuals")
plt.show()
```



4. Derivation of Coefficients for Weighted Least Square Regression

We first use a single regression model to derive coefficients for weighted least square regression.

Assume we have the following simple regression model for n observations:

$$Y_i = \beta_0 + \beta_1 X_i + e_i, \quad i = 1, \ldots, n$$

Where $e \sim N(0, \frac{\sigma^2}{w_i})$ and $w_1, \ldots, w_n$ are known positive numbers

For weighted least square regression, we are looking for β_0 and β_1 that will minimize the sum of weighted squared residuals. Here is the objective function:

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^{n} w_i (Y_i - \beta_0 - \beta_1 X_i)^2$$

We take partial derivatives of the objective function with respect to β_0 and β_1 to get the WLS estimates of $\hat{\beta}_0$ and $\hat{\beta}_1$

$$\hat{\beta}_0 = \overline{Y_w} - \hat{\beta}_1 \overline{X_w}$$
$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} w_i (X_i - \overline{X_w})(Y_i - \overline{Y_w})}{\sum_{i=1}^{n} w_i (X_i - \overline{X_w})^2}$$

Where $\overline{X_w}$ and $\overline{Y_w}$ are weighted averages of X and Y with weights w.

$$\overline{X_w} = \frac{\sum_{i=1}^{n} X_i w_i}{\sum_{i=1}^{n} w_i} \quad \text{and} \quad \overline{Y_w} = \frac{\sum_{i=1}^{n} Y_i w_i}{\sum_{i=1}^{n} w_i}$$

We can apply the same process above to solve for coefficients for weighted least square regression with multiple independent variables. This is a general form of derivation because we don't restrict the number of independent variables. Since we have multiple independent variables now, we will use the matrix form to solve the minimization problem.

For multiple weighted least regression, we have the model:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{e}$$
$$\mathrm{Var}(e) = \sigma^2 \mathbf{W}^{-1}$$

We minimize the following sum of weighted squared residuals in matrix form for multiple weighted least square regression models. Here is the objective function:

$$RSS(\boldsymbol{\beta}) = \sum_{i=1}^{n} w_i (Y_i - X_i^t \boldsymbol{\beta})^2 = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^t \mathbf{W} (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})$$

We take the partial derivatives of the above objective function with respect to $\boldsymbol{\beta}$ and set the equations to 0 to solve for $\boldsymbol{\beta}$.

$$\sum_{i=1}^{n} w_i e_i X_i^t = 0$$

The general form of WLS estimator is given by

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^t \mathbf{W} \mathbf{X})^{-1} (\mathbf{X}^t \mathbf{W} \mathbf{Y})$$

5. Conclusion

In this lesson, we first reviewed the key assumptions for an ordinary least square regression model. We then relaxed the assumption of homoscedasticity. If the data shows heteroskedasticity, which is a violation of homoscedasticity, we will use a weighted least square regression model to handle the issue.

We pointed out that OLS is a special case for WLS when the weight is 1, which means the data is unweighted. We then introduced the basics of WLS and the steps to run WLS. We finished the lesson with one example from the foreign exchange market. In the next

lesson, we will introduce another regression method when the assumptions of OLS do not hold up well in the dataset.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 2 | LESSON 3

PENALIZED REGRESSION

Reading Time	60 minutes
Prior Knowledge	OLS linear regression, Linear Algebra
Keywords	High dimensional data, Overfitting, Training dataset vs Testing dataset, Penalized Function, Shrinkage, Regularization parameter, Variance and bias trade off, K-Fold Cross Validation, Ridge Regression, Lasso Regression

In the last two lessons, we talked about how to use weighted least regression and robust regression to handle unequal variance and outlier problems. In this lesson, we are going to study ridge regression and lasso regression. These two methods are usually used to tackle high dimensional data and overfitting issues. These issues in general happen due to too many variables or when many variables are correlated. We will first discuss these data issues and then introduce ridge regression and lasso regression. We will later use the dollar index example to run ridge regression and lasso regression.

In [1]:
```
# Libraries
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.linear_model import Lasso, LassoCV, Ridge, RidgeCV
```

In [2]:
```
# Import data
data_set_r_working = pd.read_csv("M2. module_2_data.csv")
```

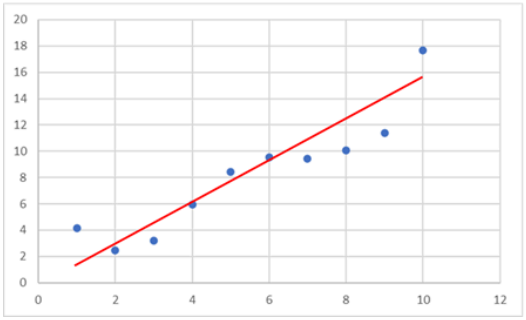
1. High Dimensional Data and Overfitting

In the field of finance, a researcher can sometimes have a large number of independent variables at hand to analyze the dependent variable. For example, in risk management, one can have 300 or 400 potential variables to run a stress test for a bank. With this large number of independent variables, it is also likely that the number of variables would exceed the number of observations. **High dimensional data** is a dataset in which the number of variables is larger than the number of observations. If a dataset has p variables and n observations and $p > n$, then this dataset is high dimensional data. In machine learning literature, independent variables are also called **predictors, covariates**, **attributes** or **features**. In this course, we will refer to them as independent variables.

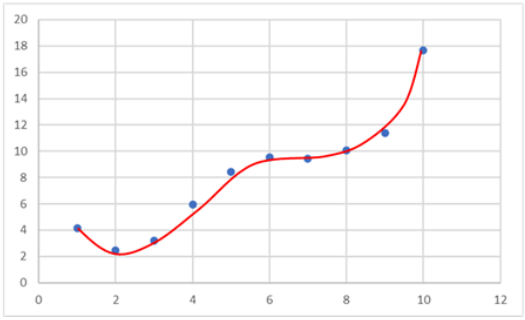
If one uses a high dimensional dataset to run a regression, the model might perfectly predict the dependent variable because there are so many independent variables in the model. While these regression models capture the signal, they also tend to incorporate the noise.

Figure 1: Simple Model vs Complicated Model

$Y = -0.66 + 1.54*X$



$Y = 10.7 + 11.4*X + 4.6*X^2+0.46*X^3+0.03*X^4$



From Figure 1, you can see there are two models to model the same data points. The model on the left is a very simple model and only has one independent variable. The model on the right is complicated and has four independent variables. This model almost perfectly fits all data points. This model not only describes the overall trend but also captures the noise. As such, the model on the right presents an **overfitting** issue. An **overfitted** model is a model not only trying to find a relationship between the dependent variable and the independent variables but also trying to model the random error. When overfitting happens, usually the coefficients of the model are far away from 0.

But what is wrong with an overfitted model? Before talking about the problem of an overfitted model, we would like to introduce two concepts first: training dataset and testing dataset. A **training dataset** is the dataset we use to build our analytical models, like regression models. A **testing dataset** is a dataset we use to check a model's performance or a model's fit *after* we build the model. In research, it is standard procedure to split the dataset available for modeling into a training dataset and a testing dataset before building a model. Then, we use the training dataset to estimate the model and use the testing dataset to test the fit of the model. The split can be any ratio, say 50:50, 30:70 or 25:75. The reason is that you can build a model that provides a perfect fit for the training dataset. However, when you use the model on the testing dataset, the model prediction of the dependent variable from testing dataset might be very poor. This is because you are trying to model all the possible information from the training dataset, including both the signal and the error. When you build an overfitted model using the training dataset and use the overfitted model to predict the testing dataset, the prediction result can be bad. If a model cannot predict well, then the model is useless.

Another issue with too many independent variables in a regression model is that these independent variables are more likely to exhibit a multicollinearity issue. When multicollinearity exists, the variance of coefficients is large. Coefficients with a large variance will make statistical inference less accurate, like hypothesis testing.

2. Penalized Regression

2.1 How Do We Tackle the Issue of Too Many Variables?

We talked about the problem of having too many variables or high dimensional data for regression analysis in the last section. What can we do about this issue?

In Module 1, we talked about using principal component analysis or factor analysis, two data dimension reduction tools to solve high dimensional data issues. You first run either PCA or factor analysis to decide what variables to keep and run the linear regression model after you decide on the independent variables from PCA or factor analysis.

In this lesson, we will introduce a different general method to handle the problem of too many variables: **penalized regression.**

2.2 Basics of Penalized Regression

When we have too many variables and use all of them to build a linear regression model, we run the risk of overfitting the model. In an OLS regression model, we try to minimize the following objective function:

$$\sum_{i=1}^{n}(Y_i - \beta_0 - \beta_1 X_{1i} - \cdots - \beta_p X_{pi})^2$$

In the OLS objective function, we can see there is no restriction on how many β (also independent variables) there are in the function. What if we assign some sort of penalty to each additional β every time we add one new independent variable? Penalized regression uses an objective function like the following:

$$\sum_{i=1}^{n}(Y_i - \beta_0 - \beta_1 X_{1i} - \cdots - \beta_p X_{pi})^2 + \lambda \sum_{j=1}^{p} f(\beta_j)$$

2.2.1 Penalty Function

$f()$ is called a **regularization function** or **penalty function**. A **penalty function** will try to pull the value of a coefficient close to 0 if the value is too large. The function basically tries to reduce the impact of an independent variable in the model if the coefficient of this independent variable is too large. It is a function that penalizes the existence of an independent variable in a model by suppressing the value of its coefficient. In an extreme case, this function may pull the coefficient to 0. When the coefficient is 0, it means the independent variable is removed from the model. The action of pulling a coefficient value towards 0 or to a certain preset value is called **shrinkage**, in that you shrink a coefficient.

There are several types of penalty functions. We are going to talk about the two most popular ones in the later sections: ridge regression and lasso regression.

2.2.2 Regularization Parameter

What is λ? λ is a **regularization parameter**. It controls the **trade-off of variance and bias** for coefficients. What does this mean? In an ideal model, we would like to achieve an accuracy for the estimated coefficients that is as high as possible and as low as possible for standard errors of such estimated coefficients.

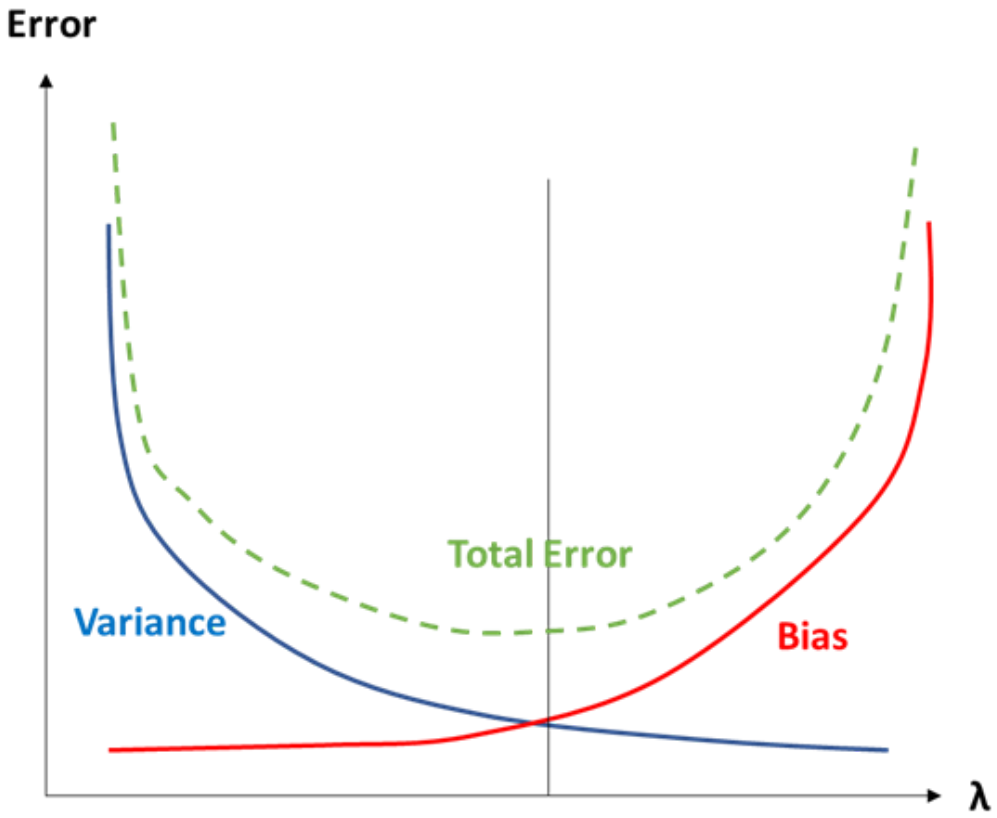
In OLS, the estimated coefficients are unbiased, which means they are accurate. However, the variance of the coefficients can be very high if there are too many variables in the model, as we argued in the previous section. An OLS objective function provides accurate coefficient estimates but not stable estimates (high variance). On the other hand, a penalty function can suppress the coefficient estimates from going too far away from 0. We can see from the penalized regression objective function that there is the original OLS objective function as well as a penalty function. An OLS objective function gives accurate coefficient estimates with high variance. With the addition of the penalty

function to the penalized regression objective function, this objective function will produce coefficient estimates with lower variance. However, the coefficient estimates from penalized regression will be biased.

Why does a penalized regression objective function produce biased coefficient estimates? Because its ability to pull coefficient estimates closer to 0 will prevent the model from fully minimizing the sum of the distances from the regression line to all data points. This is also the reason we have the penalty function in the penalized regression objective function. We don't want to put too much emphasis on the noisy part of the data.

So how can we balance the trade-off between bias and variance of the estimated coefficients? We use λ to balance the two forces. We can use a graph to illustrate the impact of the value of λ on total error.

Figure 2: Trade-Off Graph between Variance and Bias in Penalized Regression



From Figure 2, we can see that as λ goes to 0, the error from variance gets larger and the error from bias gets smaller and vice versa. We would like to find the middle point of λ that will give us the lowest total error. So how can we find the optimal λ?

2.2.3 Standardize Independent Variables

Before moving on to discuss how to find the optimal λ, there are steps we need to take to prepare the dataset before we model. The first thing is that the penalty function shouldn't include the intercept, β_0. In general, we don't penalize β_0.

Another important step to take is that of standardizing all independent variables. We talked about how the penalty function will pull the value of a coefficient closer to 0 if the value is too large. In order for this action to make sense, all independent variables need to be standardized to remove the fact that they could have different scales.

Some statistical software packages may conduct independent variable standardization automatically while running a penalized regression. Please check the manual for the specific software package you decide to use for more information.

2.2.4 K-Fold Cross Validation to Find Optimal λ

How can we find the optimal λ? We can use a method called **k-fold cross validation**. You can choose any number for k. Usually, 5 and 10 are popular numbers for k. Let's use 5-fold cross validation as an example to explain how it works.

Let's assume that λ is a value between 0 and 20. We will try each of the following λ: (0, 0.25, 0.5, 0.75, ... , 19.75, 20). There will be a total of 80 values to try for λ.

The next thing we need to do is to divide our dataset equally into five groups for 5-fold cross validation.

Say that our dataset contains 100 observations. Then, we will create 5 groups each containing 20 observations. Let's call our 5 groups A, B, C, D, and E.

We also have the penalized regression objective function:

$$\sum_{i=1}^{n}(Y_i - \beta_0 - \beta_1 X_{1i} - \cdots - \beta_p X_{pi})^2 + \lambda \sum_{j=1}^{p} f(\beta_j)$$

Now, we have our possible λ values to test, five equal groups from our data, and a penalized regression objective function. We are ready to run iterations to generate optimal λ.

Here are the steps to generate optimal λ:

1. Choose $\lambda = 0$
2. Choose A as the testing dataset and B, C, D, and E together as the training dataset.
3. Build a penalized regression model using the training dataset.
4. Plug testing dataset A into the fitted model from step 3 and calculate the sum of residuals squared from the fitted model. Keep the sum of squared residuals.
5. Go back to step 2, and this time, select the next group, B, to be the testing dataset and A, C, D, and E together as the training dataset. Now, repeat steps 3, 4, and 5 until each of the five groups has been treated as a testing dataset.
6. Calculate the average sum of squared residuals from the recorded five values of the sum of squared residuals from each of the five different testing datasets.
7. Keep a record of λ and its associated average sum of squared residuals.
8. Repeat from Step 1 to Step 7 to go through each possible value of λ
9. Now, you have 80 data points of λ and its associated average sum of squared residuals. You can find the optimal λ that gives the minimal value of the average

sum of squared residuals. The optimal λ will be used for the final penalized regression.

This is how 5-fold cross validation works. The same logic applies to k-fold cross validations. You equally divide the dataset into k groups and follow the steps above to get the optimal λ.

Most of the statistical packages can conduct k-fold cross validation for you so you don't have to do it manually. Some software will even choose optimal λ for you without any specification about the range to test for λ.

Finding a good λ is critical to running a penalized regression. Since we now know how to obtain λ, we can start to look at various types of penalized regression methods. In the following sections, we will learn ridge regression and lasso regression. These two regressions are widely used penalized regression methods.

3. Ridge Regression

In the last section, we talked about the basics of penalized regressions. We discussed regularization parameters, steps to process data, and k-fold cross validation. Now, we are going to look at various functions for a penalized regression objective function. We will start with ridge regression.

Ridge regression is one type of penalized regression. The penalty function for ridge regression is the sum of squared coefficients (betas). The mathematical formula is as follows:

$$f() = \sum_{j=1}^{p} \beta_j^2 = \|\beta\|^2$$

The above penalty function is also called an **L2 penalty function**. It is because the function is the square of the magnitude of coefficients in the model. For the detailed coefficient derivation for ridge regression, please refer to the required reading from the Penn State University, Eberly College of Science website.

Here are some key properties of ridge regression:

1. Ridge regression shrinks the coefficients of the independent variables that don't contribute very much to the model close to 0, but not 0.
2. If $\lambda = 0$, then ridge regression estimation is OLS estimation.
3. All the independent variables are still in the model.
4. Ridge regression is a good method when a researcher wants to keep all independent variables but also wants to reduce variation of estimated coefficients due to multicollinearity issue. What ridge regression does is to restrict coefficients of correlated independent variables and make their estimated values smaller.

However, if you have too many independent variables and you would like to select only a few key independent variables for your model, lasso regression is a better choice.

4. Lasso Regression

Lasso regression is short for **least absolute shrinkage and selection operator**. Lasso regression has the following penalty function:

$$f() = \sum_{j=1}^{p} |\beta_j| = \|\beta\|_1$$

The above penalty function is also called an **L1 penalty function**. It is because the function is the absolute value of the magnitude of coefficients in the model. For more technical details on lasso regression, please refer to the required reading from the Penn State University, Eberly College of Science website.

Here are some key properties of lasso regression:

1. Lasso regression can shrink the coefficients of the independent variables that don't contribute very much to the model to 0. Hence, lasso regression can remove independent variables. Lasso regression can be used for variable selection.
2. If $\lambda = 0$, then a lasso regression estimation is an OLS estimation.
3. Lasso regression is a good choice when you have too many independent variables and you are looking to select only key variables for the model. Lasso regression can first group independent variables with high correlations together, choose only one of them for the model, and drop the rest.

5. An Example: Foreign Exchange Market and Penalized Regression

We've talked a lot about ridge regression and lasso regression in the previous sections. Let's apply these two regression methods to our foreign exchange market example and see how they perform. In order to have a solid comparison for different regression methods, we'll also run an OLS regression as the base result and compare results from OLS, ridge regression and lasso regression.

We are going to expand our U.S. Dollar Index example by adding more independent variables in this section. Using the data from 2016, here is the list of our variables:

Dependent Variable:

DXY: U.S. Dollar Index daily return

Independent Variables:

- **METALS**: Gold and Silver Index daily return
- **US_STK**: S&P 500 Index daily return
- **X13W_TB**: 13-week Treasury Bills daily return
- **X10Y_TBY**: 10 Year Treasury Bond Yield daily return
- **EURUSD**: EURUSD daily return

We are going to have five independent variables in our regression models to understand their impact on the U.S. Dollar Index, DXY. We will run three regression models: OLS, ridge, and lasso.

Here are the steps to run the three models.

1. Split the data into a training dataset and a testing dataset. We split the whole dataset into 80% for training data and 20% for testing data.
2. We first run the OLS regression on the training dataset to get the OLS model. We record the OLS coefficients. Then, we plug the test dataset into the OLS model to calculate test R^2 and test root mean squared error.
3. We then run the ridge regression. We first set a range of values for λ. We then run 10-fold cross validation to get the optimal λ from the training data. Once we get the optimal λ, we run the final ridge regression on the training data and get the coefficients. Once we get the ridge regression model, we plug test dataset into ridge regression model and calculate test R^2 and test root mean squared error.
4. Lastly, we run the lasso regression. We do the same as what we did for ridge regression. We set a range of values for λ and run 10-fold cross validation to get optimal λ. We then run the final lasso regression and get the coefficients. After getting the lasso regression model, we plug in testing data to calculate test R^2 and test root mean squared error.

Figure 3: Model Comparison from OLS, Ridge Regression, and Lasso Regression

Data preparation

In [3]:
```
# Create Training Dataset and Testing Dataset

np.random.seed(11111)  # Random seed
nrow = data_set_r_working.shape[0]
train_sequence = sorted(np.random.choice(nrow, int(nrow * 0.8), replace=False))
test_sequence = sorted(set(list(range(0, nrow))) - set(train_sequence))

train = data_set_r_working.filter(items=train_sequence, axis=0)
test = data_set_r_working.filter(items=test_sequence, axis=0)

# Make sure X matrix is in matrix form and Y is in vector form
ind_var = ["METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUSD"]

train_x = train.loc[:, ind_var]
train_y = train.DXY

test_x = test.loc[:, ind_var]
test_y = test.DXY

test_tot = test.loc[:, ["DXY", "METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUS
```

OLS Regression

In [4]:
```
# OLS Regression
ols_final = sm.OLS(train_y, sm.add_constant(train_x)).fit()
print(ols_final.summary2().tables[1])  # print only coefficients
```

```
# Compute test R^2 and test mean squared error
ols_pred = ols_final.predict(sm.add_constant(test_x))
ols_pred = pd.DataFrame(ols_pred, columns=["ols_p"])
ols_actual = test.DXY

ols_rss = np.sum(np.power(ols_pred.ols_p - ols_actual, 2))
ols_tss = np.sum(np.power(ols_actual - np.mean(ols_actual), 2))
ols_rsq = 1 - (ols_rss / ols_tss)
print("\n OLS_R^2", ols_rsq)

ols_RMSE = np.sqrt(ols_rss / test.shape[0])
print(" OLS_RSME", ols_RMSE)
```

```
            Coef.  Std.Err.         t         P>|t|    [0.025    0.975]
const     0.000410  0.000310  1.321022  1.880502e-01 -0.000202  0.001022
METALS   -0.070259  0.010999 -6.387663  1.219474e-09 -0.091952 -0.048566
US_STK   -0.058324  0.040419 -1.442997  1.506340e-01 -0.138040  0.021392
X10Y_TBY  0.019521  0.013955  1.398854  1.634535e-01 -0.008002  0.047045
X13W_TB   0.000696  0.003924  0.177426  8.593591e-01 -0.007044  0.008436
EURUSD   -0.087070  0.059863 -1.454492  1.474259e-01 -0.205135  0.030995

 OLS_R^2 0.10633201319436236
 OLS_RSME 0.00381807095301603
```

Ridge Regression

In [5]:

```
# Ridge Regression

# generate a sequence of lambdas to try
lambdas = [np.power(10, i) for i in np.arange(4, -4, -0.1)]
alphas = lambdas

# Scale
# train_x_scale = scale(train_x) #In case you want to scale the variables.

# Use 10-fold Cross Validation to find optimal lambda
ridge_cv = RidgeCV(alphas=alphas, cv=10, scoring="neg_mean_squared_error")
ridge_cv.fit(train_x, train_y)

# Build final ridge regression model
ridge_final = Ridge(alpha=ridge_cv.alpha_, fit_intercept=True)
ridge_final.fit(train_x, train_y)

# Print coefficients
# print('Intercept:', ridge_final.intercept_)
print(
    "\n",
    pd.DataFrame(
        (ridge_final.coef_),
        index=["METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUSD"],
        columns=["Coef."],
    ),
)

# R squared formula and mean squared error
ridge_pred = ridge_final.predict(test_x)
ridge_actual = test.DXY
ridge_rss = np.sum(np.power(ridge_pred - ridge_actual, 2))
ridge_tss = np.sum(np.power(ridge_actual - np.mean(ridge_actual), 2))
ridge_rsq = 1 - ridge_rss / ridge_tss
```

```
print("\n Ridge_R^2", ridge_rsq)

ridge_RMSE = np.sqrt(ridge_rss / test.shape[0])
print("Ridge_RSME", ridge_RMSE)
```

```
             Coef.
METALS    -0.061364
US_STK    -0.016407
X10Y_TBY   0.015374
X13W_TB    0.001258
EURUSD    -0.012517

 Ridge_R^2 0.12384290357405447
Ridge_RSME 0.0037804794866077144
```

LASSO Regression

In [6]:
```
# LASSO Regression

# generate a sequence of lambdas to try
lambdas = [np.power(10, i) for i in np.arange(6, -6, -0.1)]

# Compile model
lasso_cv = LassoCV(cv=10, alphas=lambdas)
lasso_cv.fit(train_x, train_y)  # Fit Model

# Scale
# train_x_scale = scale(train_x) #In case you want to scale the variables.

# Build final LASSO regression model
lasso_final = Lasso(alpha=lasso_cv.alpha_, fit_intercept=True)
lasso_final.fit(train_x, train_y)

# Print results
# print('Intercept:', lasso_final.intercept_)
print(
    "\n",
    pd.DataFrame(
        (lasso_final.coef_),
        index=["METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUSD"],
        columns=["Coef."],
    ),
)

# R squared formula and mean squared error
lasso_pred = lasso_final.predict(test_x)
lasso_actual = test.DXY
lasso_rss = np.sum(np.power(lasso_pred - lasso_actual, 2))
lasso_tss = np.sum(np.power(lasso_actual - np.mean(lasso_actual), 2))
lasso_rsq = 1 - lasso_rss / lasso_tss
print("\n LASSO_R^2: ", lasso_rsq)

lasso_RMSE = np.sqrt(lasso_rss / test.shape[0])
print("LASSO_RSME: ", lasso_RMSE)
```

```
              Coef.
METALS    -0.059688
US_STK    -0.000000
X10Y_TBY   0.000000
X13W_TB    0.000000
EURUSD    -0.000000

 LASSO_R^2:  0.14151036558996777
LASSO_RSME:  0.0037421692200670896
```

All of the results in one table

In [7]:
```
# All of the results in one table

OLS_df = pd.DataFrame(ols_final.summary2().tables[1]["Coef."]).rename(
    columns={"Coef.": "OLS"}
)

Ridge_df = pd.DataFrame(
    np.insert(ridge_final.coef_, 0, ridge_final.intercept_),
    index=["Intercept", "METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUSD"],
    columns=["Ridge"],
)

Lasso_df = pd.DataFrame(
    np.insert(lasso_final.coef_, 0, lasso_final.intercept_),
    index=["Intercept", "METALS", "US_STK", "X10Y_TBY", "X13W_TB", "EURUSD"],
    columns=["Lasso"],
)

df = OLS_df.merge(Ridge_df, left_index=True, right_index=True)
df = df.merge(Lasso_df, left_index=True, right_index=True)

df = pd.concat(
    [
        df,
        pd.DataFrame(
            {
                "OLS": [ols_rsq, ols_RMSE],
                "Ridge": [ridge_rsq, ridge_RMSE],
                "Lasso": [lasso_rsq, lasso_RMSE],
            },
            index=["R sq", "Root Mean Sq. Err"],
        ),
    ]
)

display(df)
```

	OLS	Ridge	Lasso
METALS	-0.070259	-0.061364	-0.059688
US_STK	-0.058324	-0.016407	-0.000000
X10Y_TBY	0.019521	0.015374	0.000000
X13W_TB	0.000696	0.001258	0.000000
EURUSD	-0.087070	-0.012517	-0.000000
R sq	0.106332	0.123843	0.141510
Root Mean Sq. Err	0.003818	0.003780	0.003742

Figure 3 shows the model results from our three methods. The table includes coefficients, test R^2 and test root mean squared error for OLS, ridge and lasso regressions. From Figure 3, we can see lasso has the best R^2 for the testing dataset, followed by ridge with OLS being the worst. We also notice that there is only one independent variable, *METALS*, which has a non-0 coefficient in lasso regression. Lasso only selects *METALS* for the regression model. Lasso makes the model very simple, yet it has the best fit for testing data.

6. Conclusion

In this lesson, we discussed the issue of high dimensional data and why it can cause an overfitting problem for a regression model. We then introduced the idea of penalized regression to handle issues with too many variables and overfitting. We went through the basics of penalized regression and then presented the two most popular methods: ridge regression and lasso regression. We showed their penalty functions, and their properties, and their differences. Lastly, we applied ridge regression and lasso regression to our U.S. Dollar Index model example. We compared the three regression results of our U.S. Dollar Index model: OLS, ridge and lasso. In the next lesson, we will continue our journey by exploring regression analysis and learning about non-parametric regression analysis.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.





T.1.1 - Robust Regression Methods

Note! that the material in Sections 13.3-13.6 is considerably more technical than the preceding Lessons. It is offered as an introduction to this advanced topic and, given the technical nature of the material, it could be considered optional in the context of this course

The ordinary least squares estimates for linear regression are optimal when all of the regression assumptions are valid. When some of these assumptions are invalid, least squares regression can perform poorly. Residual diagnostics can help guide you to where the breakdown in assumptions occurs but can be time-consuming and sometimes difficult for the untrained eye. **Robust regression** methods provide an alternative to least squares regression by requiring less restrictive assumptions. These methods attempt to dampen the influence of outlying cases in order to provide a better fit to the majority of the data.

Outliers have a tendency to pull the least squares fit too far in their direction by receiving much more "weight" than they deserve. Typically, you would expect that the weight attached to each observation would be on average 1/*n* in a data set with *n* observations. However, outliers may receive considerably more weight, leading to distorted estimates of the regression coefficients. This distortion results in outliers which are difficult

to identify since their residuals are much smaller than they would otherwise be (if the distortion wasn't present). As we have seen, scatterplots may be used to assess outliers when a small number of predictors are present. However, the complexity added by additional predictor variables can hide the outliers from view in these scatterplots. Robust regression down-weights the influence of outliers, which makes their residuals larger and easier to identify.

For our first robust regression method, suppose we have a data set of size *n* such that

$$\begin{aligned} y_i &= \mathbf{x}_i^{\mathrm{T}}\beta + \epsilon_i \\ \Rightarrow \epsilon_i(\beta) &= y_i - \mathbf{x}_i^{\mathrm{T}}\beta, \end{aligned}$$

where $i = 1, \ldots, n$. Here we have rewritten the error term as $\epsilon_i(\beta)$ to reflect the error term's dependency on the regression coefficients. Ordinary least squares are sometimes known as **L_2-norm regression** since it is minimizing the L_2-norm of the residuals (i.e., the squares of the residuals). Thus, observations with high residuals (and high squared residuals) will pull the least squares to fit more in that direction. An alternative is to use what is sometimes known as **least absolute deviation** (or **L_1-norm regression**), which minimizes the L_1-norm of the residuals (i.e., the absolute value of the residuals). Formally defined, the least absolute deviation estimator is

$$\hat{\beta}_{\mathrm{LAD}} = \arg\min_{\beta} \sum_{i=1}^{n} |\epsilon_i(\beta)|,$$

which in turn minimizes the absolute value of the residuals (i.e., $|r_i|$).

Another quite common robust regression method falls into a class of estimators called ***M*-estimators** (and there are also other related classes such as *R*-estimators and *S*-estimators, whose properties we will not explore). *M*-estimators attempt to minimize the sum of a chosen function $\rho(\cdot)$ which is acting on the residuals. Formally defined, *M*-estimators are given by

$$\hat{\beta}_{\mathrm{M}} = \arg\min_{\beta} \sum_{i=1}^{n} \rho(\epsilon_i(\beta)).$$

The *M* stands for "maximum likelihood" since $\rho(\cdot)$ is related to the likelihood function for a suitable assumed residual distribution. Notice that, if assuming normality, then $\rho(z) = \frac{1}{2}z^2$ results in the ordinary least squares estimate.

Some *M*-estimators are influenced by the scale of the residuals, so a scale-invariant version of the *M*-estimator is used:

$$\hat{\beta}_{\mathrm{M}} = \arg\min_{\beta} \sum_{i=1}^{n} \rho\left(\frac{\epsilon_i(\beta)}{\tau}\right),$$

where τ is a measure of the scale. An estimate of τ is given by

$$\hat{\tau} = \frac{\mathrm{med}_i |r_i - \tilde{r}|}{0.6745},$$

where $\tilde{r}$ is the median of the residuals. Minimization of the above is accomplished primarily in two steps:

1. Set $\frac{\partial \rho}{\partial \beta_j} = 0$ for each $j = 0, 1, \ldots, p-1$, resulting in a set of p nonlinear equations $\sum_{i=1}^{n} x_{i,j} \psi\left(\frac{\epsilon_i}{\tau}\right) = 0$, where $\psi(\cdot) = \rho'(\cdot)$. $\psi(\cdot)$ is called the **influence function**.
2. A numerical method called **iteratively reweighted least squares** (**IRLS**) (mentioned in Section 13.1) is used to iteratively estimate the weighted least squares estimate until a stopping criterion is met. Specifically, for iterations $t = 0, 1, \ldots$

$$\hat{\beta}^{(t+1)} = (\mathbf{X}^{\mathrm{T}}(\mathbf{W}^{-1})^{(t)}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}(\mathbf{W}^{-1})^{(t)}\mathbf{y},$$

where $(\mathbf{W}^{-1})^{(t)} = \mathbf{diag}(w_1^{(t)}, \ldots, w_n^{(t)})$ such that

$$w_i^{(t)} = \begin{cases} \dfrac{\psi((y_i - \mathbf{x}_i^{\mathrm{t}}\beta^{(t)})/\hat{\tau}^{(t)})}{(y_i \mathbf{x}_i^{\mathrm{t}}\beta^{(t)})/\hat{\tau}^{(t)}}, & \text{if } y_i \neq \mathbf{x}_i^{\mathrm{T}}\beta^{(t)}; \\ 1, & \text{if } y_i = \mathbf{x}_i^{\mathrm{T}}\beta^{(t)}. \end{cases}$$

Three common functions chosen in *M*-estimation are given below:

Andrews' Sine

$$\rho(z) = \begin{cases} c[1-\cos(z/c)], & \text{if } |z| < \pi c; \\ 2c, & \text{if } |z| \geq \pi c \end{cases}$$

$$\psi(z) = \begin{cases} \sin(z/c), & \text{if } |z| < \pi c; \\ 0, & \text{if } |z| \geq \pi c \end{cases} \quad \text{where } c \approx 1.339.$$

$$w(z) = \begin{cases} \frac{\sin(z/c)}{z/c}, & \text{if } |z| < \pi c; \\ 0, & \text{if } |z| \geq \pi c, \end{cases}$$

Huber's Method

$$\rho(z) = \begin{cases} z^2, & \text{if } |z| < c; \\ |2z|c - c^2, & \text{if } |z| \geq c \end{cases}$$

$$\psi(z) = \begin{cases} z, & \text{if } |z| < c; \\ c[\text{sgn}(z)], & \text{if } |z| \geq c \end{cases} \quad \text{where } c \approx 1.345.$$

$$w(z) = \begin{cases} 1, & \text{if } |z| < c; \\ c/|z|. & \text{if } |z| \geq c, \end{cases}$$

Tukey's Biweight

$$\rho(z) = \begin{cases} \frac{c^2}{3}\left\{1-(1-(\frac{z}{c})^2)^3\right\}, & \text{if } |z| < c; \\ 2c, & \text{if } |z| \geq c \end{cases}$$

$$\psi(z) = \begin{cases} z[1-(\frac{z}{c})^2]^2, & \text{if } |z| < c; \\ 0, & \text{if } |z| \geq c \end{cases} \quad \text{where } c \approx 4.685.$$

$$w(z) = \begin{cases} [1-(\frac{z}{c})^2]^2, & \text{if } |z| < c; \\ 0, & \text{if } |z| \geq c, \end{cases}$$

« Previous Next »

Lesson

Lesson 1: Simple Linear Regression
Lesson 2: SLR Model Evaluation
Lesson 3: SLR Estimation & Prediction
Lesson 4: SLR Model Assumptions

Lesson 5: Multiple Linear Regression

Lesson 6: MLR Model Evaluation

Lesson 7: MLR Estimation, Prediction & Model Assumptions

Lesson 8: Categorical Predictors

Lesson 9: Data Transformations

Lesson 10: Model Building

Lesson 11: Influential Points

Lesson 12: Multicollinearity & Other Regression Pitfalls

Lesson 13: Weighted Least Squares & Logistic Regressions

Optional Content

Topic 1: Robust Regression

T.1.1 - Robust Regression Methods

T1.1.1 - Robust Regression Examples

T.1.2 - Resistant Regression Methods

T.1.3 - Regression Depth

Topic 2: Time Series & Autocorrelation

Topic 3: Poisson & Nonlinear Regression

Resource Menu

Notation Used in this Course

Common Procedures in Minitab